

이음_MVP_최종_DB_구조

기준: PostgreSQL 15+ / Supabase Auth / Supabase PostgreSQL

기본 시간대: `Asia/Seoul`

기준 화면: 이음 MVP 최종의 최종 화면 흐름 / 최종 Figma 사용자 흐름

핵심 원칙: 현재 MVP에서 실제 사용하는 데이터만 저장한다.

문서 상태: 화면 흐름 더블체크 완료·최종 동결본

0. 최종 확정 정책

인증·사용자

- 이메일 로그인, 카카오 로그인, 비밀번호는 `Supabase Auth` 가 관리한다.
- 앱 DB에는 비밀번호, 카카오 ID, 로그인 provider를 저장하지 않는다.
- `user_profiles.id` 는 `auth.users.id` 와 동일한 UUID를 사용한다.
- 이메일은 `auth.users.email` 에서 조회하므로 `user_profiles` 에 중복 저장하지 않는다.
- 닉네임은 중복을 허용한다.
- 닉네임 글자 수 제한은 없다.
- 공백만 있는 닉네임은 금지한다.
- 온보딩 전에는 닉네임이 `NULL` 일 수 있다.
- 실제 회원탈퇴와 비밀번호 변경은 MVP에서 구현하지 않는다.
- 로그인·회원가입의 입력칸별 오류는 프론트 입력 상태다. 앱 DB에 오류 문구나 입력 검증 결과를 저장하지 않는다.
- 이메일 형식, 비밀번호 미입력, 비밀번호 확인 불일치, 약관 미동의 등 프론트 검증에 실패하면 Supabase Auth 또는 FastAPI 요청을 보내지 않는다.
- Supabase Auth의 중복 이메일 응답은 회원가입 이메일 입력칸의 `이미 가입된 이메일이에요.` 오류로 변환해 표시하되 앱 DB에는 저장하지 않는다.

7일 계획

- 한 `planning_cycle` 은 정확히 7일이다.
- 사용자당 `ACTIVE` cycle은 최대 1개다.

- ACTIVE cycle이 존재하는 동안 새 7일 계획을 시작하는 기능은 제외한다.
- 마지막 날짜의 저녁 분기 정산이 끝나면 cycle은 ENDED가 된다.
- cycle 종료 시 남은 시간이 있는 ACTIVE Task는 모두 CANCELLED로 변경한다.
- 취소된 Task의 cancelled_at에는 cycle 종료 시각을 저장한다.
- 종료된 cycle의 Task는 다음 cycle로 자동 이월하지 않는다.
- 과거 캘린더에 필요한 planning_cycle, CheckIn, COMPLETED PlanBlock, NOT_DONE PlanBlock, 과거 고정 일정과 Task 정보는 기록으로 보존한다.
- CANCELLED Task 자체는 현재 계획과 Task 목록에서 숨긴다.
- CANCELLED Task에 연결된 과거 COMPLETED PlanBlock도 `plan_blocks.display_title` 을 그대로 표시한다.
- 과거 PlanBlock 문구는 Task의 현재 제목을 다시 조합해 만들지 않는다.
- NOT_DONE PlanBlock은 캘린더 목록에서 숨긴다.
- 다음 7일 계획은 자동 생성하지 않는다.
- 사용자가 계획관리 탭에서 직접 새 계획을 시작한다.
- Task의 원본 제목·전체 분량·예상 시간은 `tasks` 에서 관리한다.
- 각 PlanBlock은 해당 실행 단위에 배정된 분량과 시간을 별도로 저장한다.
- 각 PlanBlock은 홈·결과·캘린더에서 그대로 사용할 자연스러운 한 줄 실행 문구인 `display_title` 을 저장한다.
- `display_title` 은 Task 제목 + 분량 + 시간 을 고정 형식으로 단순 연결한 계산값이 아니라, SOLAR가 실행 단위에 맞게 확정한 표시 문구다.
- 같은 PlanBlock이 `PLANNED → CHECKED → COMPLETED` 또는 `PLANNED → NOT_DONE` 으로 상태만 바뀌면 `display_title` 은 변경하지 않는다.
- 현재·미래 PlanBlock이 재계획으로 삭제·재생성될 때만 새 실행 단위에 맞춰 `display_title` 을 다시 생성한다.
- 고정 일정은 시작 일시와 종료 일시를 저장하며 자정을 넘는 일정도 한 행으로 지원한다.

SOLAR 계획관리

- 사용자당 작성 중·실행 중·실패 처리 대기 중이거나 성공 결과를 아직 확인하지 않은 SOLAR 요청은 최대 1개다.
- 앱이 종료된 경우 작성 중 요청은 보존하고 다시 실행했을 때 복원한다.
- 계획관리 작성 중 다른 탭 또는 시스템 뒤로 가기를 선택하면 즉시 삭제하지 않고 작성 중 나가 기 팝업을 먼저 표시한다.

앱 종료
 → 요청 유지
 → 앱 재실행 시 복원

다른 탭 또는 시스템 뒤로 가기
 → 작성 중 나가기 팝업
 → 이 시점에는 DB 변경 없음

[계속 작성]
 → 요청·카드·채팅 유지
 → 계획관리 화면 유지

[내용 삭제하고 나가기]
 → solar_requests 물리 삭제
 → solar_request_items, solar_messages CASCADE 삭제
 → 삭제 성공 후 선택한 탭 또는 이전 화면으로 이동

삭제 실패
 → 요청·카드·채팅 유지
 → 이동하지 않음
 → 일반 오류 문구와 [다시 시도] 표시

- 작성 중 삭제 허용 상태는 `COLLECTING`, `CHANGE_CONFIRMATION`, `CHANGE_INPUT`, `FINAL_REVIEW` 다.
- `EXECUTING` 중에는 요청 삭제와 실행 취소를 허용하지 않는다.
- 실행 실패 시 Task·고정 일정·PlanBlock 반영은 전부 롤백한다.
- 다시 시도할 때 기존 요청과 카드를 재사용한다.
- 성공 결과 화면은 별도 결과 테이블을 만들지 않고 `solar_requests.execution_result`를 사용한다.
- Figma의 채팅 중간 Frame은 모두 하나의 `/plan-management` Route와 하나의 `COLLECTING` 상태로 구현한다.
- `COLLECTING` 내부에서는 메시지·카드·질문 데이터만 갱신하며 질문 종류별 DB 상태 Enum이나 Route를 추가하지 않는다.

화면 흐름·오류 처리 연계

다음 항목은 최종 화면 흐름과 API 계약에 포함되지만 별도 테이블이나 DB Enum을 추가하지 않는다.

마감 임박 경고

단일 `blockingNotice.task` 객체 사용 안 함
 → `blockingNotice.items` 배열 사용

경고 대상 1개
 → `items` 길이 1

경고 대상 여러 개
→ 같은 items 배열에 모두 포함

경고 확인은 다음 일괄 API를 사용한다.

POST /tasks/deadline-warnings/acknowledge

```
{  
  "taskIds": ["task-id-1", "task-id-2"]  
}
```

각 Task의 기존 `deadline_warning_acknowledged_at` 을 한 트랜잭션에서 갱신하므로 별도 경고 테이블은 만들지 않는다.

일반 오류

정보를 불러오지 못했어요.
잠시 후 다시 시도해 주세요.

[다시 시도]

조회·삭제·일반 재조회 실패는 DB 상태값으로 저장하지 않는다.

진행 상태 오류

진행 상태를 확인하지 못했어요.
작업은 계속 진행 중일 수 있어요.

[다시 확인]

EXECUTING 조회 실패
→ `solar_requests.status` 변경 없음
→ FAILED로 판단하지 않음

FINALIZING 조회 실패
→ `check_ins.finalized_at` 변경 없음
→ CheckIn 실패로 판단하지 않음

하단 탭·로딩 표현

하단 탭 표시 여부와 Route는 프론트 화면 정책이며 DB 컬럼으로 저장하지 않는다.

EXECUTING

→ 하단 탭 표시

FINALIZING

→ 하단 탭 없음

‘로딩 페이지’라는 포괄 상태

→ DB Enum과 화면 상태로 만들지 않음

상태 전이 공통 원칙

상태값과 그 상태가 요구하는 연관 컬럼은 반드시 **같은 SQL UPDATE 문 안에서 함께 변경한다.**

예:

```
planning_cycles
→ status = ENDED
→ ended_at 저장
→ 같은 UPDATE에서 처리
```

```
tasks
→ status = COMPLETED
→ remaining_minutes = 0
→ completed_at 저장
→ 같은 UPDATE에서 처리
```

```
plan_blocks
→ CHECKED에서 COMPLETED로 변경
→ check_in_id 저장
→ 같은 UPDATE에서 처리
```

```
solar_requests
→ status = COMPLETED
→ executed_at, execution_result 저장
→ error_code, error_message NULL 처리
→ 같은 UPDATE에서 처리
```

이 원칙은 상태 CHECK 제약을 위반하는 중간 상태가 DB에 생기지 않게 하기 위한 것이다.

제외 기능

다음은 MVP에서 사용하지 않는다.

반복 Task
반복 고정 일정
recurrence_group
이번 일정만 / 반복 전체
PDF 업로드
Document Parse
Information Extract
회원탈퇴 실제 처리
비밀번호 변경
ACTIVE cycle 강제 종료 후 새 cycle 생성
Task 남은 시간 전용 직접 수정 API

분기 정의

MORNING 04:00 ~ 11:59
AFTERNOON 12:00 ~ 17:59
EVENING 18:00 ~ 다음 날 03:59

오전·오후·저녁 각각 PlanBlock 기본 용량은 최대 240분이다.

저녁 분기의 자정 이후 시간은 전날 저녁으로 저장한다.

2026-07-29 02:00
→ plan_date = 2026-07-28
→ period = EVENING

서버는 날짜와 분기를 계산할 때 다음 공통 규칙을 사용한다.

resolvePlanDate(timestamp):

04:00 ~ 23:59
→ 해당 달력 날짜

00:00 ~ 03:59
→ 전날 날짜

resolvePeriod(timestamp):

04:00 ~ 11:59
→ MORNING

12:00 ~ 17:59
→ AFTERNOON

18:00 ~ 23:59 또는 00:00 ~ 03:59
→ EVENING

분기 자동 정산

- 분기 종료 시각이 되면 서버 Worker가 앱 실행 여부와 관계없이 자동 정산한다.

MORNING 종료
→ 매일 12:00

AFTERNOON 종료
→ 매일 18:00

EVENING 종료
→ 다음 날 04:00

- `GET /bootstrap` 과 `GET /home/current` 는 정산을 시작하거나 DB를 변경하지 않고 현재 상태만 조회한다.
- 정상 정산은 정시 Worker가 담당한다.
- 서버 중단이나 Worker 실패에 대비해 별도의 서버 내부 복구 Worker를 둔다.

서버 시작 시 복구 스캔
정시 Worker 실행 전·후 복구 스캔
finalized_at = NULL인 오래된 CheckIn 재처리
종료 시각이 지났지만 CheckIn이 없는 분기 탐지

- 복구 대상이 여러 개이면 시간 순서를 지키기 위해 가장 이전 장애 대상부터 처리한다.
- 정시 Worker와 복구 Worker는 같은 정산 함수, 같은 UNIQUE, 같은 행 잠금 규칙을 사용한다.
- API 조회 요청이나 사용자의 탭 이동이 정산 성공의 전제가 되지 않는다.
- ACTIVE_CYCLE 실행 시 정산 진행 또는 장애 복구 대기 상태가 감지되면 계획 변경을 시작하지 않고, 서버 내부 복구를 요청한 뒤 사용자가 정산 완료 후 다시 시도하도록 응답한다.

1. 최종 테이블 구성

Supabase Auth

auth.users

앱 DB

- 앱에서 실제 사용하는 테이블은 총 **9개**다.

plan_cycle_status

solar_request_purpose

NEW_CYCLE
ACTIVE_CYCLE

solar_request_status

COLLECTING
CHANGE_CONFIRMATION
CHANGE_INPUT
FINAL_REVIEW
EXECUTING
COMPLETED
FAILED

solar_action

CREATE
UPDATE
DELETE

solar_entity_type

TASK
FIXED_SCHEDULE

solar_item_status

INFO_MISSING
READY
EXECUTED

solar_message_role

USER
ASSISTANT

solar_message_kind

TEXT
QUESTION

ERROR
DECISION

plan_period

MORNING
AFTERNOON
EVENING

task_status

ACTIVE
COMPLETED
CANCELLED

plan_block_status

PLANNED
CHECKED
COMPLETED
NOT_DONE

estimate_source

USER
AI_ESTIMATED

amount_source

USER
AI_ESTIMATED
UNKNOWN

공통 저장 원칙

- 모든 `TIMESTAMP_TZ` 는 절대 시각으로 저장하고, 논리 날짜·분기 계산만 `Asia/Seoul` 기준으로 수행한다.

- UUID PK의 기본값은 `gen_random_uuid()` 를 사용한다. 단, `user_profiles.id` 는 `auth.users.id` 를 그대로 사용한다.
- `created_at` 기본값은 `now()` 다.
- `updated_at` 이 있는 테이블은 INSERT 시 `now()` 를 사용하고, UPDATE 트리거로 자동 갱신한다.
- 분 단위 시간은 정수 분으로 저장한다.
- Figma의 산책 10분 같은 실행 단위를 지원하기 위해 Task와 PlanBlock 시간에 30분 배수 제약을 두지 않는다.
- PlanBlock 한 개는 최소 1분, 한 분기에는 최대 240분까지 배치할 수 있다.
- 제목·문구 TEXT는 길이를 임의 제한하지 않되, 공백만 있는 값은 금지한다.
- JSONB는 화면 DTO 전체를 영구 저장하는 용도가 아니라 SOLAR 임시 카드, 질문 메타데이터, 실행 결과 요약에만 사용한다.
- 화면 Route, 탭 표시 여부, 토스트, 네트워크 오류 문구는 DB에 저장하지 않는다.

권장 기본값

```
id UUID DEFAULT gen_random_uuid()
created_at TIMESTAMPTZ DEFAULT now()
updated_at TIMESTAMPTZ DEFAULT now()
onboarding_completed BOOLEAN DEFAULT false
execution_attempt_count SMALLINT DEFAULT 0
missing_fields TEXT[] DEFAULT '{}'
metadata JSONB DEFAULT '{} '::jsonb
```

`updated_at` 공통 트리거

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$;
```

다음 테이블에 `BEFORE UPDATE` 트리거를 연결한다.

```
user_profiles
planning_cycles
solar_requests
solar_request_items
tasks
fixed_schedules
plan_blocks
```

4. user_profiles

Supabase Auth 사용자와 앱 프로필을 1:1로 연결한다.

컬럼	타입	설명
id	UUID PK/FK	auth.users.id
nickname	TEXT NULL	온보딩 전에는 NULL 가능
onboarding_completed	BOOLEAN NOT NULL	닉네임 설정 완료 여부
created_at	TIMESTAMPTZ NOT NULL	생성 시각
updated_at	TIMESTAMPTZ NOT NULL	수정 시각

정책

- 이메일은 auth.users.email 에서 조회한다.
- 앱 프로필 테이블에 이메일을 중복 저장하지 않는다.
- 실제 비밀번호와 카카오 로그인 정보도 auth.users 가 관리한다.
- 닉네임에는 UNIQUE 제약을 두지 않는다.
- 닉네임 글자 수 제한은 없다.

상태·조건 제약

```
onboarding_completed = false
→ nickname은 NULL 가능
```

```
onboarding_completed = true
→ nickname IS NOT NULL
→ 공백 문자열이 아니어야 함
```

```
CHECK (
  nickname IS NULL
  OR btrim(nickname) <> ''
)
```

```
CHECK (
  onboarding_completed = false
  OR nickname IS NOT NULL
)
```

5. planning_cycles

사용자의 실제 7일 계획 범위를 저장한다.

컬럼	타입	설명
id	UUID PK	cycle ID
user_id	UUID FK NOT NULL	user_profiles.id
start_date	DATE NOT NULL	7일 계획 시작일
end_date	DATE NOT NULL	시작일 포함 7번째 날
status	plan_cycle_status NOT NULL	ACTIVE, ENDED
activated_at	TIMESTAMPTZ NOT NULL	실제 활성화 시각
ended_at	TIMESTAMPTZ NULL	논리적 cycle 종료 시각
created_at	TIMESTAMPTZ NOT NULL	생성 시각
updated_at	TIMESTAMPTZ NOT NULL	수정 시각

정책

- 사용자당 ACTIVE cycle은 최대 1개다.
- start_date 는 resolvePlanDate(activated_at) 으로 정한다.
- 기간은 항상 정확히 7일이다.
- end_date = start_date + 6일 이다.
- 논리상 cycle 종료 시각은 end_date 다음 날 04:00이다.
- cycle은 물리 삭제하지 않는다.
- 종료된 cycle은 ENDED 상태로 보존한다.

- ACTIVE cycle이 있을 때 새로운 cycle을 만드는 기능은 지원하지 않는다.

```
cycle_end_at
= end_date 다음 날 04:00 Asia/Seoul

status = ENDED
→ ended_at = cycle_end_at
```

백그라운드 정산이나 오류 복구가 04:00보다 늦게 실행되더라도 ended_at에는 실제 처리 시각인 now()가 아니라 cycle_end_at을 저장한다.

상태 전이

```
ACTIVE
→ 마지막 날짜 EVENING의 정시 정산 Worker 완료
  또는 장애 복구 Worker 완료
→ ENDED
```

다음 역방향 전이는 금지한다.

```
ENDED → ACTIVE
```

상태·조건 제약

```
start_date = resolvePlanDate(activated_at)
end_date = start_date + 6일
```

resolvePlanDate(activated_at) 검증은 시간대 계산이 필요하므로 서버 트랜잭션에서 보장한다.

```
status = ACTIVE
→ ended_at IS NULL
```

```
status = ENDED
→ ended_at IS NOT NULL
```

필수 UNIQUE

```
CREATE UNIQUE INDEX uq_planning_cycles_one_active_per_user
ON planning_cycles(user_id)
WHERE status = 'ACTIVE';
```

6. solar_requests

사용자가 계획관리에서 진행하는 한 번의 SOLAR 입력·수정·실행 단위를 저장한다.

컬럼	타입	설명
id	UUID PK	SOLAR 요청 ID
user_id	UUID FK NOT NULL	요청 사용자
plan_cycle_id	UUID FK NULL	수정 대상 또는 실행 후 생성된 cycle
purpose	solar_request_purpose NOT NULL	새 7일 또는 현재 계획 수정
status	solar_request_status NOT NULL	현재 진행 단계
raw_input	TEXT NOT NULL	최초 사용자 입력
current_item_order	SMALLINT NULL	현재 질문 중인 카드 순서
confirmed_at	TIMESTAMP TZ NULL	최종 검토 확정 시각
execution_started_at	TIMESTAMP TZ NULL	실행 시작 시각
executed_at	TIMESTAMP TZ NULL	실행 성공 완료 시각
execution_attempt_count	SMALLINT NOT NULL	실행·재시도 횟수
execution_result	JSONB NULL	성공 결과 화면 데이터
error_code	TEXT NULL	실행 실패 코드
error_message	TEXT NULL	실행 실패 문구
result_acknowledged_at	TIMESTAMP TZ NULL	성공 결과 확인 시각
created_at	TIMESTAMP TZ NOT NULL	생성 시각
updated_at	TIMESTAMP TZ NOT NULL	수정 시각

plan_cycle_id 의미

purpose = ACTIVE_CYCLE
→ 처음부터 수정 대상 ACTIVE cycle ID 저장

```
purpose = NEW_CYCLE
AND 실행 전
→ plan_cycle_id = NULL
```

```
purpose = NEW_CYCLE
AND 실행 성공
→ 새로 생성된 cycle ID 저장
```

execution_result 예시

성공 화면에 필요한 생성·수정·삭제 건수를 저장한다.

```
{
  "createdTaskCount": 2,
  "updatedTaskCount": 1,
  "cancelledTaskCount": 1,
  "createdFixedScheduleCount": 1,
  "updatedFixedScheduleCount": 0,
  "deletedFixedScheduleCount": 1
}
```

별도 성공 결과 테이블을 만들지 않는다.

화면의 요약 건수는 다음과 같이 계산한다.

```
taskCount
= createdTaskCount
+ updatedTaskCount
+ cancelledTaskCount
```

```
fixedScheduleCount
= createdFixedScheduleCount
+ updatedFixedScheduleCount
+ deletedFixedScheduleCount
```

화면:

할 일 taskCount건 · 고정 일정 fixedScheduleCount건을 반영했어요.

서버가 합계 필드를 추가로 계산해 반환해도 되지만, 원본 세부 건수는 `execution_result`에 유지한다.

실행 중복 방지

별도의 `execution_idempotency_key` 는 사용하지 않는다.

`solar_requests.id` 인 `requestId` 를 실행 단위 식별자로 사용한다.

같은 `requestId` 로 `execute` 가 중복 호출됨

- `solar_requests` 의 현재 `status` 확인
- `FINAL_REVIEW` 상태일 때만 `EXECUTING` 으로 변경
- 이미 `EXECUTING`, `COMPLETED`, `FAILED` 라면
새로운 실행 작업을 등록하지 않고 현재 상태를 반환

`FINAL_REVIEW` → `EXECUTING` 변경은 조건부 `UPDATE` 또는 행 잠금을 이용해 한 번만 성공하도록 처리한다.

```
UPDATE solar_requests
SET
    status = 'EXECUTING',
    execution_started_at = now(),
    execution_attempt_count = execution_attempt_count + 1,
    executed_at = NULL,
    execution_result = NULL,
    error_code = NULL,
    error_message = NULL,
    updated_at = now()
WHERE
    id = :request_id
    AND status = 'FINAL_REVIEW'
RETURNING id, status;
```

`UPDATE` 결과가 1행이면 백그라운드 실행 작업을 등록한다.

`UPDATE` 결과가 0행이면 이미 상태가 변경된 요청이므로 새로운 실행 작업을 등록하지 않고 현재 요청 상태를 반환한다.

`execution_attempt_count` 는 요청 생성 시 0으로 시작한다.

최초 `execute` 시작

- 0에서 1로 증가

`retry` 시작

- 기존 값에서 1 증가

`retry` 는 새로운 SOLAR 요청을 만들지 않고 같은 `solar_requests` 행과 같은 `solar_request_items` 를 재사용한다.

```
UPDATE solar_requests
SET
    status = 'EXECUTING',
    execution_started_at = now(),
    execution_attempt_count = execution_attempt_count + 1,
    executed_at = NULL,
    execution_result = NULL,
    error_code = NULL,
    error_message = NULL,
    updated_at = now()
WHERE
    id = :request_id
    AND status = 'FAILED'
RETURNING id, status;
```

`retry` UPDATE 결과가 1행일 때만 새로운 백그라운드 실행 작업을 등록한다.

정책

- 사용자당 현재 요청은 최대 1개다.
- 작성 중 이탈 시 프론트는 확인 팝업을 먼저 표시하며, 팝업 표시만으로는 DB를 변경하지 않는다.

```
삭제 허용 상태:
COLLECTING
CHANGE_CONFIRMATION
CHANGE_INPUT
FINAL_REVIEW

[내용 삭제하고 나가기] 선택
→ DELETE /solar/requests/{requestId}
→ 요청 물리 삭제
→ 카드·메시지 CASCADE 삭제

EXECUTING
→ 삭제 및 취소 불가
```

- DELETE 실패 시 요청·카드·메시지를 유지하고 화면을 이동하지 않는다.
- 앱만 종료된 경우에는 요청을 삭제하지 않는다.
- 실행 성공 요청은 결과 확인 이후 이력으로 남길 수 있다.

- 실행 실패 요청은 재시도하거나 사용자가 취소하여 삭제한다.
- `COLLECTING`의 분석·질문·재질문은 모두 같은 요청 행과 같은 `/plan-management` Route에서 처리한다.
- 질문 종류마다 새로운 `solar_request_status`를 추가하지 않는다.

상태 전이

COLLECTING
→ 모든 카드 READY
→ CHANGE_CONFIRMATION

CHANGE_CONFIRMATION
→ 네
→ CHANGE_INPUT

CHANGE_CONFIRMATION
→ 아니요
→ FINAL_REVIEW

CHANGE_INPUT
→ 정보 부족 발생
→ COLLECTING

CHANGE_INPUT
→ 모든 카드 READY
→ CHANGE_CONFIRMATION

FINAL_REVIEW
→ 수정
→ CHANGE_INPUT

FINAL_REVIEW
→ 모두 등록
→ EXECUTING

EXECUTING
→ 성공

→ COMPLETED

EXECUTING

→ 실패

→ FAILED

FAILED

→ 다시 시도

→ EXECUTING

상태·조건 제약

EXECUTING

```
status = EXECUTING
→ execution_started_at IS NOT NULL
→ execution_attempt_count >= 1
→ executed_at IS NULL
→ execution_result IS NULL
→ error_code IS NULL
→ error_message IS NULL
```

COMPLETED

```
status = COMPLETED
→ execution_started_at IS NOT NULL
→ execution_attempt_count >= 1
→ executed_at IS NOT NULL
→ execution_result IS NOT NULL
→ error_code IS NULL
→ error_message IS NULL
```

FAILED

```
status = FAILED
→ execution_started_at IS NOT NULL
→ execution_attempt_count >= 1
→ executed_at IS NULL
→ execution_result IS NULL
→ error_code IS NOT NULL
→ error_message IS NOT NULL
```

그리고 `execution_attempt_count` 공통 제약도 추가해.

```
execution_attempt_count >= 0
```

```
result_acknowledged_at IS NOT NULL  
→ status = COMPLETED
```

```
CHECK (btrim(raw_input) <> '')
```

```
CHECK (execution_attempt_count >= 0)
```

프론트 화면 계산

```
status = COMPLETED  
AND result_acknowledged_at IS NULL  
→ EXECUTION_SUCCESS
```

```
status = FAILED  
→ EXECUTION_FAILED
```

```
현재 요청 없음  
AND ACTIVE cycle 있음  
→ ACTIVE_CYCLE_ENTRY
```

```
현재 요청 없음  
AND ACTIVE cycle 없음  
→ NEW_CYCLE_ENTRY
```

`EXECUTION_SUCCESS`, `EXECUTION_FAILED`, `ACTIVE_CYCLE_ENTRY`, `NEW_CYCLE_ENTRY` 는 DB enum이 아니다.

현재 요청 UNIQUE

```
CREATE UNIQUE INDEX uq_solar_requests_one_current_per_user  
ON solar_requests(user_id)  
WHERE  
    status IN (  
        'COLLECTING',
```

```

        'CHANGE_CONFIRMATION',
        'CHANGE_INPUT',
        'FINAL_REVIEW',
        'EXECUTING',
        'FAILED'
    )
    OR (
        status = 'COMPLETED'
        AND result_acknowledged_at IS NULL
    );

```

7. solar_request_items

SOLAR가 사용자 입력을 Task 카드와 고정 일정 카드로 분리한 임시 데이터다.

컬럼	타입	설명
id	UUID PK	카드 ID
user_id	UUID FK NOT NULL	소유 사용자
solar_request_id	UUID FK NOT NULL	부모 SOLAR 요청
item_order	SMALLINT NOT NULL	카드 순서
action	solar_action NOT NULL	생성·수정·삭제
entity_type	solar_entity_type NOT NULL	Task 또는 고정 일정
status	solar_item_status NOT NULL	정보 부족·준비·반영 완료
raw_line_text	TEXT NOT NULL	원본 한 줄 입력
normalized_payload	JSONB NOT NULL	구조화된 카드 데이터
missing_fields	TEXT[] NOT NULL	부족한 필드 목록
pending_question	JSONB NULL	현재 질문 내용
target_task_id	UUID FK NULL	수정·삭제 대상 Task
target_fixed_schedule_id	UUID FK NULL	수정·삭제 대상 고정 일정
executed_at	TIMESTAMPTZ NULL	실제 반영 시각
created_at	TIMESTAMPTZ NOT NULL	생성 시각
updated_at	TIMESTAMPTZ NOT NULL	수정 시각

normalized_payload 예시

Task 카드

```
{
  "title": "자료구조 과제",
  "deadlineAt": "2026-07-31T23:59:59+09:00",
  "estimatedMinutes": 180,
  "estimatedMinutesSource": "USER",
  "amountText": "문제 5개",
  "amountSource": "USER"
}
```

고정 일정 카드

```
{
  "title": "알바",
  "startAt": "2026-08-01T22:00:00+09:00",
  "endAt": "2026-08-02T02:00:00+09:00"
}
```

정책

- 여러 줄 입력을 카드 단위로 나눈다.
- 가장 위의 `INFO_MISSING` 카드 하나만 질문한다.
- 이미 `READY` 인 카드에는 다시 질문하지 않는다.
- 수정 대상이 여러 개라 애매하면 특정 카드를 자동 선택하지 않는다.
- SOLAR가 사용자에게 구체적으로 다시 입력해 달라고 질문한다.
- 반복 범위 선택 컬럼은 존재하지 않는다.
- 실행 성공 후 카드 상태를 `EXECUTED` 로 변경한다.
- `COLLECTING` 은 하나의 `/plan-management` Route에서 처리한다.
- 화면 응답의 `requestItems` 는 이 테이블의 카드 목록을 `item_order` 순서로 변환한 값이다.
- 화면 응답의 `pendingItemId` 는 `current_item_order` 와 카드 목록을 이용해 계산하거나 현재 `pending_question` 대상 카드 ID로 반환한다.
- 화면 응답의 `currentQuestion` , `quickReplies` , `inputPlaceholder` 는 `pending_question` 과 메시지 메타데이터로 구성한다.
- 위 화면 응답 필드는 별도 DB 컬럼이나 별도 상태 Enum으로 추가하지 않는다.

대상 무결성

```
action = CREATE
→ target_task_id IS NULL
→ target_fixed_schedule_id IS NULL
```

```
action IN (UPDATE, DELETE)
AND entity_type = TASK
→ target_task_id IS NOT NULL
→ target_fixed_schedule_id IS NULL
```

```
action = UPDATE
AND entity_type = FIXED_SCHEDULE
→ target_task_id IS NULL
→ target_fixed_schedule_id IS NOT NULL
```

```
action = DELETE
AND entity_type = FIXED_SCHEDULE
AND status != EXECUTED
→ target_task_id IS NULL
→ target_fixed_schedule_id IS NOT NULL
```

```
action = DELETE
AND entity_type = FIXED_SCHEDULE
AND status = EXECUTED
→ 대상 fixed_schedules 행이 물리 삭제되므로
   target_fixed_schedule_id = NULL을 허용
```

고정 일정 DELETE 대상 검증은 실행 전에 서버에서 수행하며, 실행 완료 후에는 ON DELETE SET NULL을 허용한다.

카드 상태 제약

```
status = INFO_MISSING
→ missing_fields 길이 > 0
→ executed_at IS NULL
```



```
status = READY
→ missing_fields = 빈 배열
→ executed_at IS NULL
```

```
status = EXECUTED
→ missing_fields = 빈 배열
→ executed_at IS NOT NULL
```

필수 UNIQUE

```
CREATE UNIQUE INDEX uq_solar_request_items_order
ON solar_request_items(solar_request_id, item_order);
```

```
CREATE UNIQUE INDEX uq_tasks_source_request_item
ON tasks(source_request_item_id)
WHERE source_request_item_id IS NOT NULL;
```

```
CREATE UNIQUE INDEX uq_fixed_schedules_source_request_item
ON fixed_schedules(source_request_item_id)
WHERE source_request_item_id IS NOT NULL;
```

8. solar_messages

SOLAR 채팅을 앱 재실행 후 복원하기 위한 메시지 테이블이다.

컬럼	타입	설명
id	UUID PK	메시지 ID
user_id	UUID FK NOT NULL	소유 사용자
solar_request_id	UUID FK NOT NULL	부모 SOLAR 요청
client_event_id	TEXT NULL	중복 전송 방지용 ID
sequence_no	INTEGER NOT NULL	채팅 표시 순서
role	solar_message_role NOT NULL	USER 또는 ASSISTANT
kind	solar_message_kind NOT NULL	일반·질문·오류·결정
content	TEXT NOT NULL	표시 문구
metadata	JSONB NOT NULL	관련 카드·버튼 선택 정보

컬럼	타입	설명
<code>created_at</code>	TIMESTAMP TZ NOT NULL	생성 시각

정책

- `sequence_no` 순서대로 채팅을 복원한다.
- 사용자 메시지는 `client_event_id` 로 중복 저장을 막는다.
- 서버가 생성한 ASSISTANT 메시지는 `client_event_id = NULL` 이어도 된다.
- SOLAR 분석 실패 문구와 재시도 메시지도 저장할 수 있다.
- 부모 요청 삭제 시 메시지도 함께 삭제한다.
- `COLLECTING` 화면의 `messages` 는 이 테이블을 `sequence_no` 순서로 조회해 반환한다.
- 질문 종류별 Route는 만들지 않으며, 빠른 선택 버튼·질문 대상 카드·입력창 placeholder 등 화면 보조 정보는 `metadata` 와 `solar_request_items.pending_question` 으로 구성한다.
- 분석 중 화면 잠금이나 로딩 표시는 프론트 UI 상태이며 별도 메시지 상태 Enum이나 DB 컬럼으로 저장하지 않는다.

```
CHECK (btrim(content) <> '')
```

필수 UNIQUE

```
CREATE UNIQUE INDEX uq_solar_messages_sequence
ON solar_messages(solar_request_id, sequence_no);
```

```
CREATE UNIQUE INDEX uq_solar_messages_client_event
ON solar_messages(solar_request_id, client_event_id)
WHERE client_event_id IS NOT NULL;
```

9. tasks

사용자가 실제로 해야 하는 할 일을 저장한다.

컬럼	타입	설명
<code>id</code>	UUID PK	Task ID
<code>user_id</code>	UUID FK NOT NULL	소유 사용자
<code>plan_cycle_id</code>	UUID FK NOT NULL	소속 7일 cycle

컬럼	타입	설명
<code>source_request_item_id</code>	UUID FK NULL	생성 출처 카드
<code>title</code>	TEXT NOT NULL	할 일 제목
<code>deadline_at</code>	TIMESTAMPTZ NULL	마감 일시
<code>amount_text</code>	TEXT NULL	분량 설명
<code>amount_source</code>	<code>amount_source</code> NOT NULL	사용자·AI 추정·모름
<code>initial_minutes</code>	INTEGER NOT NULL	최초 예상 시간
<code>estimated_minutes</code>	INTEGER NOT NULL	현재 전체 예상 시간
<code>estimated_minutes_source</code>	<code>estimate_source</code> NOT NULL	사용자 입력 또는 AI 추정
<code>remaining_minutes</code>	INTEGER NOT NULL	실제 남은 시간
<code>status</code>	<code>task_status</code> NOT NULL	활성·완료·취소
<code>deadline_warning_acknowledged_at</code>	TIMESTAMPTZ NULL	마감 경고 확인 시각
<code>completed_at</code>	TIMESTAMPTZ NULL	완료 시각
<code>cancelled_at</code>	TIMESTAMPTZ NULL	취소 시각
<code>created_at</code>	TIMESTAMPTZ NOT NULL	생성 시각
<code>updated_at</code>	TIMESTAMPTZ NOT NULL	수정 시각

시간 컬럼 의미

`initial_minutes`
 = 최초 확정 예상 시간
 = 생성 후 변경하지 않음

`estimated_minutes`
 = 현재 전체 작업 예상 시간
 = SOLAR 수정 요청으로 변경 가능

`remaining_minutes`
 = 현재 실제로 남은 시간
 = 분기 정산 또는 SOLAR 수정으로 변경

정산 전 실제 남은 시간 계산

현재 분기에서 CHECKED 상태지만 아직 CheckIn 정산이 끝나지 않은 시간은 `remaining_minutes`에 아직 반영되지 않는다.

```
unfinalized_checked_minutes
= 해당 Task의 현재 분기 CHECKED PlanBlock 시간 합
```

```
effective_remaining_minutes
= GREATEST(
    0,
    remaining_minutes - unfinalized_checked_minutes
)
```

effective_remaining_minutes는 DB 컬럼으로 저장하지 않고
마감 경고와 현재 **cycle** 재계획 시 서버에서 계산한다.

생성 규칙

```
Task 최초 생성
→ initial_minutes
= estimated_minutes
= remaining_minutes
```

마감 규칙

사용자가 날짜와 시간을 모두 입력한 경우:
deadline_at = 입력한 날짜와 시간

사용자가 날짜만 입력한 경우:
deadline_at = 입력한 날짜 23:59:59

처음에는 날짜를 모른다고 했지만, 재질문 후 날짜만 입력한 경우:
deadline_at = 입력한 날짜 23:59:59

처음에도 날짜를 모르고, 재질문 후에도 날짜를 모르는 경우:
deadline_at = NULL

```
deadline_at = NULL
→ 마감 임박 경고 대상 제외
→ 마감이 존재하는 Task보다 낮은 우선순위로 배치
```

날짜를 입력한 경우에는 마감 시각을 다시 묻지 않고 해당 날짜의 23:59:59를 적용한다.

즉:

금요일까지 자료구조 과제
→ 금요일 23:59:59

cycle 범위 밖 마감 처리

Task의 실제 마감이 현재 7일 cycle 범위 밖이어도 등록을 허용한다.

실제 마감은 deadline_at에 그대로 저장한다.

예:
현재 cycle: 7월 28일 ~ 8월 3일
실제 마감: 8월 15일 18:00

deadline_at
= 8월 15일 18:00

현재 cycle 안에서의 배치 우선순위 계산에는 다음 계산값을 사용한다.

cycle_end_at
= end_date 다음 날 04:00 Asia/Seoul

```
CASE
  WHEN deadline_at IS NULL THEN NULL
  WHEN deadline_at < cycle_end_at
    THEN deadline_at
  ELSE cycle_end_at
END
```

여기서 cycle 종료 시각은:

end_date 다음 날 04:00

이다.

실제 마감이 cycle 종료 시각 이후인 Task
→ 현재 cycle에서는 논리적 cycle 종료 시각인
end_date 다음 날 04:00 마감처럼 계산
→ 2주 뒤 마감이라는 이유로 지나치게 적은 시간을 배정하지 않음

마지막 날짜의 자정 이후 마감이더라도 04:00 전이라면
현재 cycle의 마지막 **EVENING**에 포함되므로 실제 마감 시각을 유지한다.

예:

```
cycle end_date = 8월 3일
Task 마감 = 8월 4일 02:00

→ 8월 3일 EVENING 범위 안
→ planning_deadline_at = 8월 4일 02:00
```

planning_deadline_at은 DB 컬럼으로 저장하지 않고 서버에서 계산한다.

이렇게 하면:

```
내일 마감 Task
→ 먼저 배치

2주 뒤 마감 Task
→ cycle 종료 시각인 end_date 다음 날 04:00을
  배치용 마감으로 사용

마감 모름 Task
→ 가장 낮은 우선순위
```

상태 전이

```
ACTIVE
→ remaining_minutes = 0
→ COMPLETED
```

```
ACTIVE
→ 사용자가 삭제 요청
→ CANCELLED
```

```
ACTIVE
→ 소속 planning_cycle 종료 시 remaining_minutes > 0
→ CANCELLED
```

금지:

```
COMPLETED → ACTIVE
CANCELLED → ACTIVE
```

상태·조건 제약

Figma와 화면 흐름의 `산책 10분` 같은 실행 단위를 지원하기 위해 30분 배수 제약을 두지 않는다.

```
CHECK (initial_minutes >= 1)
CHECK (estimated_minutes >= 1)
CHECK (remaining_minutes >= 0)
```

```
status = ACTIVE
→ remaining_minutes > 0
→ completed_at IS NULL
→ cancelled_at IS NULL
```

```
status = COMPLETED
→ remaining_minutes = 0
→ completed_at IS NOT NULL
→ cancelled_at IS NULL
```

```
status = CANCELLED
→ remaining_minutes >= 0
→ completed_at IS NULL
→ cancelled_at IS NOT NULL
```

```
CHECK (btrim(title) <> '')
```

분량 출처와 텍스트는 다음을 맞춘다.

```
amount_source = UNKNOWN
→ amount_text IS NULL
```

```
amount_source IN (USER, AI_ESTIMATED)
→ amount_text IS NOT NULL
→ btrim(amount_text) <> ''
```

`remaining_minutes <= estimated_minutes` 제약은 두지 않는다. 사용자가 SOLAR를 통해 실제 남은 작업 시간을 늘릴 수 있기 때문이다.

Task 제목이나 전체 분량이 수정돼도 과거 PlanBlock의 표시 문구는 변경하지 않는다. 현재·미래 PlanBlock이 실제로 재계획되어 새 행으로 생성될 때만 새로운 `plan_blocks.display_title` 과 `allocated_amount_text` 를 사용한다.

마감 경고 확인·초기화

마감 임박 경고는 별도 테이블을 만들지 않고 각 Task의 `deadline_warning_acknowledged_at` 으로 관리한다.

일괄 확인

```
POST /tasks/deadline-warnings/acknowledge
```

요청의 `taskIds` 에 포함된 Task들을 한 트랜잭션에서 처리한다.

1. `taskIds` 중복 제거
2. 모든 Task가 요청 사용자 소유인지 검증
3. 대상 Task 행 잠금
4. `deadline_warning_acknowledged_at = now()` 일괄 저장
5. 트랜잭션 커밋

- 이미 확인된 Task가 다시 포함되어도 성공으로 처리할 수 있도록 멍등하게 구현한다.
- 다른 사용자의 Task ID가 포함되면 전체 요청을 거부하고 어떤 행도 변경하지 않는다.
- 화면에는 현재 `blockingNotice.items` 에 표시된 모든 `taskId` 를 전송한다.

확인 상태 초기화

다음 정보가 의미 있게 변경되면:

```
deadline_at  
remaining_minutes  
estimated_minutes
```

서버는 다음과 같이 처리한다.

```
deadline_warning_acknowledged_at = NULL
```

즉, 위 세 값 중 하나라도 의미 있게 변경되면 이전 경고 확인 기록을 초기화하고, 경고 조건을 다시 계산한다.

10. `fixed_schedules`

사용자가 반드시 지켜야 하는 정확한 시작·종료 일정을 저장한다.

컬럼	타입	기본값	설명
<code>id</code>	UUID PK	<code>gen_random_uuid()</code>	고정 일정 ID
<code>user_id</code>	UUID FK NOT NULL		소유 사용자
<code>plan_cycle_id</code>	UUID FK NOT NULL		소속 7일 cycle
<code>source_request_item_id</code>	UUID FK NULL	<code>NULL</code>	생성 출처 카드
<code>title</code>	TEXT NOT NULL		일정 제목
<code>start_at</code>	TIMESTAMPTZ NOT NULL		시작 일시
<code>end_at</code>	TIMESTAMPTZ NOT NULL		종료 일시
<code>created_at</code>	TIMESTAMPTZ NOT NULL	<code>now()</code>	생성 시각
<code>updated_at</code>	TIMESTAMPTZ NOT NULL	<code>now()</code>	수정 시각

정책

- 시작 날짜·시각과 종료 날짜·시각을 정확히 저장한다.
- 자정을 넘는 일정도 하나의 `fixed_schedules` 행으로 저장한다.

금요일 22:00 ~ 토요일 02:00 알바
 → `start_at` = 금요일 22:00
 → `end_at` = 토요일 02:00
 → 행 1개

- 오전부터 오후, 저녁부터 다음 날 새벽처럼 여러 분기에 걸쳐도 행을 분할하지 않는다.
- 캘린더 API는 한 행과 겹치는 날짜·분기마다 표시용 구간을 계산해 반환할 수 있으나, DB에는 원본 일정 한 행만 보존한다.
- PlanBlock 배치 가능 시간 계산에서는 각 분기와 겹치는 실제 구간을 분 단위로 계산한다.
- 여러 고정 일정이 겹치면 점유 시간은 합집합으로 계산해 중복 차감하지 않는다.
- 고정 일정끼리 겹치는 입력 자체는 허용한다.
- 홈 화면에는 고정 일정 카드를 표시하지 않는다.
- 고정 일정을 PlanBlock으로 변환하지 않는다.
- 체크 대상이 아니며 진행률과 CheckIn 점수에서 제외한다.
- 반복 일정은 지원하지 않는다.
- 수정·삭제는 `start_at > 현재 시각` 인 미래 일정만 허용한다.
- 사용자가 삭제한 미래 고정 일정은 물리 삭제한다.
- 이미 시작했거나 종료된 일정은 수정·삭제하지 않고 과거 기록으로 보존한다.

논리 날짜·cycle 범위

고정 일정이 자정을 넘을 수 있으므로 시작일과 종료일이 같은 달력 날짜여야 한다는 제약은 두지 않는다.

```
start_plan_date
= resolvePlanDate(start_at)

end_plan_date
= resolvePlanDate(end_at - 1 microsecond)
```

다음은 모두 만족해야 한다.

```
planning_cycle.start_date
<= start_plan_date
<= planning_cycle.end_date

planning_cycle.start_date
<= end_plan_date
<= planning_cycle.end_date
```

종료 시각에서 아주 작은 값을 빼는 이유는 04:00 처럼 정확히 분기 경계에서 끝나는 일정이 다음 논리 날짜를 점유한 것으로 잘못 계산되지 않게 하기 위해서다.

상태·조건 제약

```
CHECK (start_at < end_at)
CHECK (btrim(title) <> '')
```

cycle 범위 검증은 다른 테이블과 Asia/Seoul 시간대 계산이 필요하므로 서버 트랜잭션 또는 DB 트리거에서 보장한다.

11. plan_blocks

Task를 날짜와 오전·오후·저녁의 실제 실행 단위로 나눈 계획 블록이다.

컬럼

컬럼	타입	기본값	설명
id	UUID PK	gen_random_uuid()	PlanBlock ID
user_id	UUID FK NOT NULL		소유 사용자
plan_cycle_id	UUID FK NOT NULL		소속 cycle

컬럼	타입	기본값	설명
<code>task_id</code>	UUID FK NOT NULL		대상 Task
<code>plan_date</code>	DATE NOT NULL		논리상 계획 날짜
<code>period</code>	<code>plan_period</code> NOT NULL		오전·오후·저녁
<code>allocated_minutes</code>	INTEGER NOT NULL		해당 블록에 배정된 시간
<code>allocated_amount_text</code>	TEXT NULL	<code>NULL</code>	해당 블록에 배정된 분량 문구
<code>display_title</code>	TEXT NOT NULL		화면에 표시할 자연스러운 한 줄 실행 문구
<code>display_order</code>	SMALLINT NOT NULL		같은 분기 안의 표시 순서
<code>status</code>	<code>plan_block_status</code> NOT NULL	<code>'PLANNED'</code>	계획·체크·완료·미완료
<code>checked_at</code>	TIMESTAMP TZ NULL	<code>NULL</code>	사용자가 체크한 시각
<code>check_in_id</code>	UUID FK NULL	<code>NULL</code>	정산 CheckIn
<code>rescheduled_from_block_id</code>	UUID FK NULL	<code>NULL</code>	재배치 원본 블록
<code>created_at</code>	TIMESTAMP TZ NOT NULL	<code>now()</code>	생성 시각
<code>updated_at</code>	TIMESTAMP TZ NOT NULL	<code>now()</code>	수정 시각

구조화 데이터와 표시 문구의 역할

DB에서는 실행 정보를 분리해 저장한다.

```

Task 원본 제목
→ tasks.title

Task 전체 분량
→ tasks.amount_text

해당 PlanBlock 배정 분량
→ plan_blocks.allocated_amount_text

해당 PlanBlock 배정 시간
→ plan_blocks.allocated_minutes

화면 한 줄 실행 문구
→ plan_blocks.display_title

```

예:

```
tasks.title = "자료구조"
tasks.amount_text = "6문제"

plan_blocks.allocated_amount_text = "2문제"
plan_blocks.allocated_minutes = 60
plan_blocks.display_title = "자료구조 2문제"
```

다른 예:

```
tasks.title = "산책"
plan_blocks.allocated_amount_text = NULL
plan_blocks.allocated_minutes = 10
plan_blocks.display_title = "산책 10분"
```

```
tasks.title = "영단어 암기"
plan_blocks.allocated_amount_text = NULL
plan_blocks.allocated_minutes = 30
plan_blocks.display_title = "영단어 암기"
```

`display_title` 은 조회 시 `tasks.title` , 분량, 시간을 기계적으로 다시 합쳐 만드는 계산값이 아니다. SOLAR가 PlanBlock 생성 시 실행 단위에 맞게 확정하고 DB에 저장하는 표시용 스냅샷이다.

- 분량이 실행에 도움이 되면 자연스럽게 포함할 수 있다.
- 시간 자체가 실행 단위이면 자연스럽게 포함할 수 있다.
- 분량이나 시간을 표시하지 않아도 명확하면 행동 문구만 저장할 수 있다.
- 홈, `CHECK_IN_RESULT` , 캘린더는 모두 같은 `display_title` 을 사용한다.
- `allocated_amount_text` 와 `allocated_minutes` 는 구조화 데이터이며, `display_title` 의 문자열을 파싱해 복원하지 않는다.
- 진행률과 CheckIn 점수는 PlanBlock 개수로 계산한다.
- Task 남은 시간 차감은 `allocated_minutes` 를 기준으로 한다.

생성·병합·재계획 정책

- 정확한 시작 시각은 저장하지 않는다.
- `plan_date + period + allocated_minutes` 로 분기 단위 계획을 저장한다.
- 같은 Task·날짜·분기의 PlanBlock은 한 행으로 합친다.
- 합쳐지는 경우 최종 `allocated_minutes` , `allocated_amount_text` 를 기준으로 `display_title` 도 다시 확정한다.

- PlanBlock 한 행은 최소 1분이며 최대 240분이다.
- 현재 분기의 미체크 `PLANNED` 와 미래 `PLANNED` 는 재계획 시 삭제·재생성할 수 있다.
- 재생성된 PlanBlock은 새 실행 단위에 맞는 `allocated_amount_text` 와 `display_title` 을 새로 가진다.
- 현재 분기의 `CHECKED` PlanBlock은 이동하거나 삭제하지 않는다.
- 현재 분기에 이미 `CHECKED` PlanBlock이 있는 Task에는 같은 날짜·분기의 새 PlanBlock을 추가하지 않는다.
- 해당 Task의 추가 남은 시간은 미래 분기에만 배치한다.
- 과거 `COMPLETED` , `NOT_DONE` 은 수정하거나 삭제하지 않는다.
- Task가 `CANCELLED` 가 되어도 현재 분기의 기존 `CHECKED` PlanBlock은 유지한다.
- CANCELLED Task의 현재 `CHECKED` PlanBlock은 분기 종료 시 `COMPLETED` 로 정산될 수 있다.
- CANCELLED Task의 현재 미체크 `PLANNED` 와 미래 `PLANNED` 만 삭제한다.
- Task 제목·분량·남은 시간이 수정됐다는 이유만으로 기존 PlanBlock의 `display_title` 을 문자열 치환하지 않는다.
- 현재·미래 계획이 실제로 재계산되어 PlanBlock이 삭제·재생성될 때만 새 `display_title` 을 사용한다.

표시 문구 유지 규칙

PLANNED → CHECKED
 CHECKED → PLANNED
 CHECKED → COMPLETED
 PLANNED → NOT_DONE
 → 같은 행의 `display_title`,
 `allocated_amount_text`,
 `allocated_minutes` 유지

현재·미래 PLANNED 재계획
 → 기존 행 삭제 가능
 → 새 PlanBlock 생성
 → 새 `display_title` 확정

과거 COMPLETED·NOT_DONE
 → 당시 `display_title` 영구 유지
 → 현재 `tasks.title`로 다시 조합하지 않음

별도의 `task_title_snapshot` 이나 완료 시점 제목 복사 컬럼은 필요하지 않다.

`plan_blocks.display_title` 자체가 해당 실행 단위의 확정 표시 문구다.

PlanBlock 내용 필드는 INSERT 후 직접 수정하지 않는 것을 원칙으로 한다.

```
task_id
plan_date
period
allocated_minutes
allocated_amount_text
display_title
```

재계획으로 내용이 달라지면 기존 수정 가능 `PLANNED` 행을 삭제하고 새 행을 생성한다. 체크·정산 UPDATE에서는 상태 관련 컬럼만 변경한다. 이 불변 규칙은 서버 서비스 계층 또는 `BEFORE UPDATE` 트리거로 보장한다.

체크 상태 전이

```
PLANNED
→ 사용자가 현재 분기 홈에서 체크
→ CHECKED
```

```
CHECKED
→ 현재 분기 종료 전 체크 해제
→ PLANNED
```

분기 정산 상태 전이

```
CHECKED
→ COMPLETED
```

```
PLANNED
→ NOT_DONE
```

정산 후에는 역방향 전이를 허용하지 않는다.

체크·체크 해제·정산 시 `status`, `checked_at`, `check_in_id` 를 같은 UPDATE에서 변경한다. 표시 문구와 배정 정보는 변경하지 않는다.

상태·조건 제약

```

CHECK (allocated_minutes >= 1)
CHECK (allocated_minutes <= 240)
CHECK (display_order >= 0)
CHECK (btrim(display_title) <> '')
CHECK (
    allocated_amount_text IS NULL
    OR btrim(allocated_amount_text) <> ''
)

```

```

status = PLANNED
→ checked_at IS NULL
→ check_in_id IS NULL

```

```

status = CHECKED
→ checked_at IS NOT NULL
→ check_in_id IS NULL

```

```

status = COMPLETED
→ checked_at IS NOT NULL
→ check_in_id IS NOT NULL

```

```

status = NOT_DONE
→ checked_at IS NULL
→ check_in_id IS NOT NULL

```

분기 용량 제약

분기 기본 용량은 240분이다. Figma의 **산책 10분** 같은 실행 단위를 지원하므로 최종 가용 시간을 30분 단위로 내림하지 않고 실제 정수 분 단위로 계산한다.

고정 일정 점유 시간

```

fixed_schedule_occupied_minutes
= 해당 분기와 겹치는 고정 일정 구간 합집합의 실제 분 수

```

고정 일정 여러 개가 서로 겹치면 중복 구간은 한 번만 차감한다.

미래 분기

```

available_period_minutes
= GREATEST(
    0,
    240 - fixed_schedule_occupied_minutes
)

```

```

SUM(해당 미래 분기의 PLANNED.allocated_minutes)
<= available_period_minutes

```

현재 분기

```

checked_minutes
= 현재 분기의 CHECKED PlanBlock 시간 합

full_period_fixed_occupied_minutes
= 현재 분기 전체에서 고정 일정이 점유하는 구간 합집합

```

```

remaining_capacity_by_limit
= GREATEST(
    0,
    240
    - checked_minutes
    - full_period_fixed_occupied_minutes
)

```

현재 시각 이후 실제 남은 시간도 계산한다.

```

remaining_clock_minutes
= 현재 시각부터 현재 분기 종료까지 남은 실제 분 수

future_fixed_occupied_minutes
= 현재 시각부터 분기 종료까지 고정 일정이 점유할 구간 합집합

```

```

remaining_clock_available_minutes
= GREATEST(
    0,
    remaining_clock_minutes
    - future_fixed_occupied_minutes
)

```

```

available_new_planned_minutes
= LEAST(
    remaining_capacity_by_limit,

```



```
remaining_clock_available_minutes
)
```

```
SUM(현재 분기의 새 PLANNED.allocated_minutes)
<= available_new_planned_minutes
```

현재 **CHECKED** 시간은 240분 총용량에서 차감하지만, 현재 시각 이후 새 **PLANNED** 합계에 다시 포함하지 않는다.

서버 검증 순서

1. 대상 논리 날짜와 분기를 계산한다.
2. 고정 일정과 분기의 실제 겹침 구간을 조회한다.
3. 겹침 구간 합집합을 분 단위로 계산한다.
4. 현재 분기와 미래 분기를 구분한다.
5. 현재 분기이면 실제 남은 시간도 반영한다.
6. 생성하려는 **PLANNED** 합계가 가용 시간을 초과하지 않는지 검증한다.
7. 성공한 경우에만 PlanBlock을 저장한다.
8. 최종 블록 단위에 맞는 allocated_amount_text와 display_title을 확정한다.

가용 시간이 1분 미만이면 새 PlanBlock을 생성하지 않는다.

필수 UNIQUE

```
CREATE UNIQUE INDEX uq_plan_blocks_task_date_period
ON plan_blocks(task_id, plan_date, period);
```

```
CREATE UNIQUE INDEX uq_plan_blocks_display_order
ON plan_blocks(plan_cycle_id, plan_date, period, display_order);
```

12. check_ins

분기 종료 시 PlanBlock 결과와 점수를 저장한다.

컬럼	타입	기본값	설명
id	UUID PK	gen_random_uuid()	CheckIn ID
user_id	UUID FK NOT NULL		소유 사용자
plan_cycle_id	UUID FK NOT NULL		대상 cycle
check_date	DATE NOT NULL		논리상 분기 날짜

컬럼	타입	기본값	설명
<code>period</code>	<code>plan_period</code> NOT NULL		정산 분기
<code>total_plan_count</code>	INTEGER NULL	<code>NULL</code>	전체 PlanBlock 개수
<code>completed_plan_count</code>	INTEGER NULL	<code>NULL</code>	완료 PlanBlock 개수
<code>score</code>	SMALLINT NULL	<code>NULL</code>	0~100점
<code>replan_unplaced_minutes</code>	INTEGER NULL	<code>NULL</code>	남은 분기에 배치하지 못한 시간
<code>finalization_started_at</code>	TIMESTAMPTZ NOT NULL	<code>now()</code>	정산 시작 시각
<code>replanned_at</code>	TIMESTAMPTZ NULL	<code>NULL</code>	재계획 완료 시각
<code>finalized_at</code>	TIMESTAMPTZ NULL	<code>NULL</code>	전체 정산 완료 시각
<code>result_acknowledged_at</code>	TIMESTAMPTZ NULL	<code>NULL</code>	결과 확인 시각
<code>created_at</code>	TIMESTAMPTZ NOT NULL	<code>now()</code>	생성 시각

FINALIZING 복원

분기 종료 시 Worker가 정산을 시작하면 CheckIn 행을 먼저 생성한다.

```
finalized_at IS NULL
→ 정산 시작 또는 정산 복구 중
→ 홈 FINALIZING
```

정산이 끝나면 다음 값을 저장한다.

```
total_plan_count
completed_plan_count
score
replan_unplaced_minutes
replanned_at
finalized_at
```

해당 분기에 PlanBlock이 하나도 없으면 CheckIn을 생성하지 않는다.

정산 도중 실패해 `finalized_at = NULL` 인 행이 남으면 새 CheckIn을 만들지 않고 동일 행을 잠가 복구한다. 이 복구는 서버 장애에 대한 백엔드 안전장치이며 정상 사용자 흐름은 정시 Worker 자동 정산이다.

정책

- `cycle + 논리 날짜 + 분기` 당 CheckIn은 최대 1개다.
- 고정 일정은 PlanBlock 개수와 점수에서 제외한다.
- 점수는 시간 비율이 아니라 PlanBlock 개수 비율이다.
- 정산 당시 개수와 점수를 저장하므로 이후 재계획이 과거 점수를 바꾸지 않는다.
- CheckIn 상세 목록은 `plan_blocks.check_in_id` 로 연결된 `COMPLETED`, `NOT_DONE` PlanBlock에서 조회한다.
- 결과 카드에는 각 PlanBlock의 저장된 `display_title` 을 그대로 사용한다.

미확인 결과 표시 정책

미확인 결과가 하나이면 해당 CheckIn을 표시한다.

미확인 결과가 여러 개이면 가장 최근에 정산된 CheckIn 하나만 표시한다.

```
SELECT *
FROM check_ins
WHERE user_id = :user_id
      AND finalized_at IS NOT NULL
      AND result_acknowledged_at IS NULL
ORDER BY finalized_at DESC, id DESC
LIMIT 1;
```

- 홈에는 선택된 CheckIn 하나의 상세 결과만 표시한다.
- 과거 미확인 결과를 결과 화면으로 연속 표시하지 않는다.
- 모든 완료된 CheckIn은 확인 여부와 관계없이 캘린더 점수로 조회할 수 있다.

결과 확인 범위

```
POST /check-ins/{checkInId}/acknowledge
```

`checkInId` 는 화면에 표시된 가장 최근 미확인 CheckIn이다.

대상이 아직 미확인 상태이면 다음 범위만 같은 `result_acknowledged_at` 으로 확인 처리한다.

```
finalized_at < 대상.finalized_at
OR (
  finalized_at = 대상.finalized_at
```

```
AND id <= 대상.id  
)
```

즉:

화면에 표시된 결과
+ 그보다 오래된 미확인 결과
→ 함께 확인

화면 표시 이후 새로 생성된 더 최근 결과
→ 이번 요청으로 확인하지 않음

권장 처리 순서:

1. 대상 CheckIn을 사용자 소유권과 함께 잠근다.
2. finalized_at IS NOT NULL인지 확인한다.
3. 대상이 이미 확인됐으면 기존 확인 결과를 반환하고 종료한다.
4. 대상의 finalized_at과 id를 경계값으로 고정한다.
5. 동일 사용자의 미확인 CheckIn 중 경계값 이하만 일괄 UPDATE한다.
6. 모든 대상에 같은 result_acknowledged_at을 저장한다.

중복 acknowledge 요청은 멍등하게 처리하며, 중복 호출 뒤 생성된 새 CheckIn은 변경하지 않는다.

점수 계산

```
score  
= ROUND(  
    completed_plan_count * 100.0  
    / total_plan_count  
)
```

total_plan_count는 1 이상인 경우에만 계산한다.

정수 나눗셈을 사용하지 않는다.

소수점이 정확히 0.5이면 올림하는 일반 반올림을 사용한다.

예:

```
1 / 8 → 13점  
2 / 3 → 67점  
5 / 7 → 71점
```

화면 상태 계산

```
finalized_at IS NULL  
→ FINALIZING
```

```
finalized_at IS NOT NULL  
AND result_acknowledged_at IS NULL  
→ CHECK_IN_RESULT 후보
```

```
result_acknowledged_at IS NOT NULL  
→ 홈 결과 확인 완료
```

상태·조건 제약

정산 중:

```
finalized_at IS NULL  
→ total_plan_count NULL 가능  
→ completed_plan_count NULL 가능  
→ score NULL 가능
```

정산 완료:

```
total_plan_count > 0  
0 <= completed_plan_count <= total_plan_count  
0 <= score <= 100  
replan_unplaced_minutes >= 0
```

```
finalized_at IS NOT NULL  
→ finalized_at >= finalization_started_at
```

```
result_acknowledged_at IS NOT NULL  
→ finalized_at IS NOT NULL  
→ result_acknowledged_at >= finalized_at
```

필수 UNIQUE

```
CREATE UNIQUE INDEX uq_check_ins_cycle_date_period  
ON check_ins(plan_cycle_id, check_date, period);
```

13. FK 삭제 정책

자식 FK	부모	삭제 정책
<code>user_profiles.id</code>	<code>auth.users.id</code>	ON DELETE RESTRICT
각 테이블의 <code>user_id</code>	<code>user_profiles.id</code>	ON DELETE RESTRICT
cycle 관련 FK	<code>planning_cycles.id</code>	ON DELETE RESTRICT
<code>solar_request_items.solar_request_id</code>	<code>solar_requests.id</code>	ON DELETE CASCADE
<code>solar_messages.solar_request_id</code>	<code>solar_requests.id</code>	ON DELETE CASCADE
<code>tasks.source_request_item_id</code>	<code>solar_request_items.id</code>	ON DELETE SET NULL
<code>fixed_schedules.source_request_item_id</code>	<code>solar_request_items.id</code>	ON DELETE SET NULL
<code>solar_request_items.target_task_id</code>	<code>tasks.id</code>	ON DELETE RESTRICT
<code>solar_request_items.target_fixed_schedule_id</code>	<code>fixed_schedules.id</code>	ON DELETE SET NULL
<code>plan_blocks.task_id</code>	<code>tasks.id</code>	ON DELETE RESTRICT
<code>plan_blocks.check_in_id</code>	<code>check_ins.id</code>	ON DELETE RESTRICT
<code>plan_blocks.rescheduled_from_block_id</code>	<code>plan_blocks.id</code>	ON DELETE SET NULL

물리 삭제 허용

작성 중 취소한 `solar_request`
 해당 요청의 `solar_request_items`
 해당 요청의 `solar_messages`
 재계획 대상인 현재 분기의 미체크 PLANNED PlanBlock
 재계획 대상 미래 PLANNED PlanBlock
 사용자가 취소한 미래 고정 일정

물리 삭제 금지

`planning_cycle`
 완료·미완료가 확정된 과거 PlanBlock
 CheckIn
 완료 Task

14. 필수 INDEX

홈·캘린더 PlanBlock 조회

홈 현재 분기와 캘린더 날짜 범위 조회가 같은 선두 컬럼을 사용하므로 인덱스 하나를 공유한다.

```
CREATE INDEX ix_plan_blocks_user_date_period
ON plan_blocks(
    user_id,
    plan_date,
    period,
    status,
    display_order
);
```

고정 일정 범위 조회

```
CREATE INDEX ix_fixed_schedules_range
ON fixed_schedules(
    user_id,
    start_at,
    end_at
);
```

자정을 넘는 일정도 동일한 범위 조건으로 조회한다.

```
start_at < 조회 종료 시각
AND end_at > 조회 시작 시각
```

마감 임박 Task 조회

```
CREATE INDEX ix_tasks_active_deadline
ON tasks(user_id, plan_cycle_id, deadline_at)
WHERE
    status = 'ACTIVE'
```

```
AND remaining_minutes > 0
AND deadline_at IS NOT NULL
AND deadline_warning_acknowledged_at IS NULL;
```

최신 미확인 CheckIn 조회

```
CREATE INDEX ix_check_ins_latest_unacknowledged
ON check_ins(
    user_id,
    finalized_at DESC,
    id DESC
)
WHERE
    finalized_at IS NOT NULL
    AND result_acknowledged_at IS NULL;
```

정산 중 CheckIn 조회

```
CREATE INDEX ix_check_ins_finalizing
ON check_ins(user_id, finalization_started_at DESC)
WHERE finalized_at IS NULL;
```

cycle에 속한 Task 조회

```
CREATE INDEX ix_tasks_plan_cycle
ON tasks(plan_cycle_id);
```

cycle에 속한 고정 일정 조회

```
CREATE INDEX ix_fixed_schedules_plan_cycle
ON fixed_schedules(plan_cycle_id);
```

cycle 단위 PlanBlock 정산·재계획

```
CREATE INDEX ix_plan_blocks_cycle_date_period
ON plan_blocks(
    plan_cycle_id,
```



```
plan_date,  
period,  
status  
);
```

CheckIn 결과 PlanBlock 조회

```
CREATE INDEX ix_plan_blocks_check_in  
ON plan_blocks(check_in_id, display_order)  
WHERE check_in_id IS NOT NULL;
```

SOLAR 요청 조회

```
CREATE INDEX ix_solar_requests_user_updated  
ON solar_requests(user_id, updated_at DESC);
```

다음 UNIQUE INDEX가 카드·메시지 순서 조회 인덱스 역할도 함께 수행하므로 별도 중복 인덱스를 만들지 않는다.

```
uq_solar_request_items_order  
uq_solar_messages_sequence
```

SOLAR 실행 중복 방지

별도의 실행 중복 방지 키나 UNIQUE INDEX를 추가하지 않는다.

```
solar_requests.id  
+ 조건부 상태 UPDATE  
+ 요청 행 잠금  
+ requestId advisory lock  
+ source_request_item_id UNIQUE  
+ 사용자당 ACTIVE cycle UNIQUE
```

위 조합으로 실행 작업·Task·고정 일정·cycle 중복 생성을 방지한다.

15. 소유자·cycle·날짜 일치 제약

모든 핵심 테이블에 `user_id` 를 저장하기 때문에 서로 다른 사용자의 부모·자식 행이 연결되지 않도록 해야 한다.

권장 방식:

복합 FK + Supabase RLS

15-1. 복합 FK를 위한 부모 UNIQUE

PostgreSQL에서 복합 FK를 연결하려면 부모 쪽 컬럼 조합에도 UNIQUE가 필요하다.

```
ALTER TABLE planning_cycles  
ADD CONSTRAINT uq_planning_cycles_id_user  
UNIQUE (id, user_id);
```

```
ALTER TABLE solar_requests  
ADD CONSTRAINT uq_solar_requests_id_user  
UNIQUE (id, user_id);
```

```
ALTER TABLE solar_request_items  
ADD CONSTRAINT uq_solar_request_items_id_user  
UNIQUE (id, user_id);
```

```
ALTER TABLE tasks  
ADD CONSTRAINT uq_tasks_id_user  
UNIQUE (id, user_id);
```

```
ALTER TABLE tasks  
ADD CONSTRAINT uq_tasks_id_user_cycle  
UNIQUE (id, user_id, plan_cycle_id);
```

```
ALTER TABLE fixed_schedules  
ADD CONSTRAINT uq_fixed_schedules_id_user  
UNIQUE (id, user_id);
```

```
ALTER TABLE plan_blocks  
ADD CONSTRAINT uq_plan_blocks_id_user_cycle_task  
UNIQUE (id, user_id, plan_cycle_id, task_id);
```

```
ALTER TABLE check_ins
ADD CONSTRAINT uq_check_ins_full_identity
UNIQUE (id, user_id, plan_cycle_id, check_date, period);
```

15-2. 사용자 소유권 일치

다음 관계는 부모와 자식의 `user_id` 가 반드시 같아야 한다.

```
solar_requests.plan_cycle_id → planning_cycles
solar_request_items.solar_request_id → solar_requests
solar_messages.solar_request_id → solar_requests
tasks.plan_cycle_id → planning_cycles
fixed_schedules.plan_cycle_id → planning_cycles
plan_blocks.plan_cycle_id → planning_cycles
check_ins.plan_cycle_id → planning_cycles
tasks.source_request_item_id → solar_request_items
fixed_schedules.source_request_item_id → solar_request_items
solar_request_items.target_task_id → tasks
solar_request_items.target_fixed_schedule_id → fixed_schedules
```

예:

```
FOREIGN KEY (plan_cycle_id, user_id)
REFERENCES planning_cycles(id, user_id)
```

같은 관계에 대해 ID 단일 FK와 복합 FK를 중복 생성하지 않는다.
소유권 검증이 필요한 관계는 복합 FK 하나로 구현한다.

복합 FK에서 `ON DELETE SET NULL` 사용

복합 FK에 `user_id` 가 포함되어 있어도 `user_id` 는 NULL로 바꾸면 안 된다.
PostgreSQL 15의 컬럼 지정형 `SET NULL` 을 사용해
nullable ID 컬럼만 NULL로 변경한다.

```
FOREIGN KEY (source_request_item_id, user_id)
REFERENCES solar_request_items(id, user_id)
ON DELETE SET NULL (source_request_item_id)
```

같은 방식으로 다음 nullable FK에 적용한다.

```
tasks.source_request_item_id
fixed_schedules.source_request_item_id
```

```
solar_request_items.target_fixed_schedule_id  
plan_blocks.rescheduled_from_block_id
```

15-3. PlanBlock과 Task의 cycle 일치

PlanBlock은 같은 사용자인데서 같은 cycle에 속한 Task만 참조할 수 있다.

```
FOREIGN KEY (  
    task_id,  
    user_id,  
    plan_cycle_id  
)  
REFERENCES tasks(  
    id,  
    user_id,  
    plan_cycle_id  
)
```

즉 다음을 항상 만족해야 한다.

```
plan_blocks.user_id = tasks.user_id  
plan_blocks.plan_cycle_id = tasks.plan_cycle_id
```

15-4. PlanBlock과 CheckIn의 분기 일치

PlanBlock이 CheckIn에 연결될 때는 사용자·cycle·날짜·분기가 모두 같아야 한다.

```
FOREIGN KEY (  
    check_in_id,  
    user_id,  
    plan_cycle_id,  
    plan_date,  
    period  
)  
REFERENCES check_ins(  
    id,  
    user_id,  
    plan_cycle_id,  
    check_date,  
    period  
)
```

```
    period
)
```

즉 다음을 항상 만족해야 한다.

```
plan_blocks.user_id = check_ins.user_id
plan_blocks.plan_cycle_id = check_ins.plan_cycle_id
plan_blocks.plan_date = check_ins.check_date
plan_blocks.period = check_ins.period
```

15-5. cycle 날짜 범위

다음 조건은 다른 테이블의 날짜를 함께 확인해야 하므로 서버 트랜잭션 또는 DB 트리거로 검증한다.

```
planning_cycles.start_date
<= plan_blocks.plan_date
<= planning_cycles.end_date
```

```
planning_cycles.start_date
<= check_ins.check_date
<= planning_cycles.end_date
```

```
resolvePlanDate(fixed_schedules.start_at)과
resolvePlanDate(fixed_schedules.end_at - 1 microsecond)
```

→ 연결된 planning_cycle의
start_date ~ end_date 범위 안

15-6. 재배치 원본 검증

재배치 원본은 같은 사용자·cycle·Task에 속한 PlanBlock만 참조할 수 있다.

```
FOREIGN KEY (
    rescheduled_from_block_id,
    user_id,
    plan_cycle_id,
    task_id
)
REFERENCES plan_blocks(
    id,
```

```

        user_id,
        plan_cycle_id,
        task_id
    )
    ON DELETE SET NULL (rescheduled_from_block_id)

```

`rescheduled_from_block_id`가 존재하면 다음을 만족해야 한다.

```

원본 PlanBlock.status = NOT_DONE
원본과 새 PlanBlock의 user_id 동일
원본과 새 PlanBlock의 plan_cycle_id 동일
원본과 새 PlanBlock의 task_id 동일
새 PlanBlock의 날짜·분기가 원본보다 미래
자기 자신을 rescheduled_from_block_id로 참조할 수 없음

```

이 검증은 서버 트랜잭션 또는 DB 트리거로 처리한다.

16. DB 상태에서 계획관리 화면 계산

계획관리의 실제 Route는 하나다.

```
/plan-management
```

DB 상태와 계산 화면은 다음과 같다.

```

solar_requests.status = COLLECTING
→ COLLECTING

```

```

solar_requests.status = CHANGE_CONFIRMATION
→ CHANGE_CONFIRMATION

```

```

solar_requests.status = CHANGE_INPUT
→ CHANGE_INPUT

```

```

solar_requests.status = FINAL_REVIEW
→ FINAL_REVIEW

```

```
solar_requests.status = EXECUTING
```

→ EXECUTING

solar_requests.status = COMPLETED
AND result_acknowledged_at IS NULL
→ EXECUTION_SUCCESS

solar_requests.status = FAILED
→ EXECUTION_FAILED

현재 요청 없음
AND ACTIVE cycle 있음
→ ACTIVE_CYCLE_ENTRY

현재 요청 없음
AND ACTIVE cycle 없음
→ NEW_CYCLE_ENTRY

따라서 다음은 DB Enum으로 만들지 않는다.

NEW_CYCLE_ENTRY
ACTIVE_CYCLE_ENTRY
EXECUTION_SUCCESS
EXECUTION_FAILED

COLLECTING 단일 화면 계산

다음 과정은 모두 같은 `solar_requests.status = COLLECTING` 으로 표현한다.

최초 입력 분석
여러 카드 표시
예상 시간 질문
분량 질문
마감 질문
고정 일정 시작·종료 시각 질문
모호한 입력 재질문
다음 정보 부족 카드로 이동

다음과 같은 중간 상태 Enum은 만들지 않는다.

ANALYZING
TASK_QUESTION
DURATION_QUESTION

AMBIGUOUS
FIXED_SCHEDULE_QUESTION

화면 DTO는 기존 테이블에서 계산한다.

```
messages
→ solar_messages를 sequence_no 순서로 조회

requestItems
→ solar_request_items를 item_order 순서로 조회

pendingItemId
→ solar_requests.current_item_order에 해당하는 카드 ID
또는 pending_question 대상 카드 ID

currentQuestion
quickReplies
inputPlaceholder
→ solar_request_items.pending_question
+ solar_messages.metadata를 이용해 구성
```

CHANGE_INPUT 에서 새로운 정보 부족이 발생하면:

```
solar_requests.status
CHANGE_INPUT → COLLECTING

Route
/plan-management 유지

화면
messages.requestItems·질문 데이터만 갱신
```

하단 탭·오류의 DB 영향

```
EXECUTING
→ 하단 탭 표시
→ DB 컬럼 추가 없음

EXECUTING 상태 조회 실패
→ solar_requests.status 유지
→ FAILED로 변경하지 않음
```

Route, 하단 탭 표시 여부, 내부 로딩 애니메이션은 DB에 저장하지 않는다.

17. DB 상태에서 홈 화면 계산

서버는 기본 화면 상태인 **homeMode** 와 선택적인 **blockingNotice** 를 계산한다.

화면 표시 우선순위는 화면 흐름과 동일하게 다음 순서로 고정한다.

1. FINALIZING
2. CHECK_IN_RESULT
3. DEADLINE_WARNING blockingNotice
4. NO_ACTIVE_CYCLE
5. NO_PLANS
6. IN_PROGRESS

1순위: 분기 정산 중

```
finalized_at IS NULL인 CheckIn 존재  
→ FINALIZING
```

네트워크 조회 실패만으로 CheckIn을 실패 상태로 변경하지 않는다.

정상 흐름에서 CheckIn 정산은 분기 종료 시각의 서버 Worker가 앱 접속 여부와 무관하게 시작한다. `GET /bootstrap`, `GET /home/current` 는 현재 상태를 조회한다.

서버 장애로 `finalized_at = NULL` 상태가 남은 경우의 복구는 백엔드 운영 안전장치이며 별도의 홈 상태 Enum을 추가하지 않는다.

2순위: 미확인 분기 결과

```
finalized_at IS NOT NULL  
AND result_acknowledged_at IS NULL인 CheckIn 존재  
→ CHECK_IN_RESULT
```

표시 대상은 가장 최근 결과 한 개다.

```
SELECT *  
FROM check_ins  
WHERE user_id = :user_id  
      AND finalized_at IS NOT NULL  
      AND result_acknowledged_at IS NULL  
ORDER BY finalized_at DESC, id DESC  
LIMIT 1;
```

`GET /home/current` 는 선택된 CheckIn의 점수와 `check_in_id` 로 연결된 `COMPLETED`, `NOT_DONE` PlanBlock을 `display_order` 순서로 반환한다. 각 카드의 기본 문구는 `plan_blocks.display_title` 이다.

3순위: 마감 임박 경고

기본 `homeMode` 를 계산한 뒤 경고 대상 Task가 하나 이상이면 다음 차단 안내를 함께 반환한다.

```
blockingNotice.type = DEADLINE_WARNING  
blockingNotice.items = 모든 경고 대상 Task 목록
```

`DEADLINE_WARNING` 은 DB Enum이나 저장된 홈 상태가 아니다.

4순위: 7일 계획 없음

```
ACTIVE planning_cycle 없음  
→ NO_ACTIVE_CYCLE
```

단, ACTIVE cycle이 없어도 미확인 CheckIn이 있으면 2순위 결과를 먼저 표시한다.

5순위: 현재 분기 일정 없음

```
ACTIVE cycle 있음  
현재 분기 PLANNED·CHECKED PlanBlock 없음  
→ NO_PLANS
```

현재 분기에 고정 일정만 있어도 홈에는 고정 일정을 표시하지 않으므로 `NO_PLANS` 다.

6순위: 현재 분기 진행

```
ACTIVE cycle 있음  
현재 분기 PLANNED 또는 CHECKED PlanBlock 있음  
→ IN_PROGRESS
```

홈 DTO:

```
PlanBlock 목록  
→ plan_date + period의 PLANNED·CHECKED  
→ display_order 순  
→ display_title 표시  
  
완료 개수  
→ CHECKED 개수  
  
전체 개수  
→ PLANNED + CHECKED 개수
```

진행률

→ CHECKED 개수 / 전체 개수 × 100

CANCELLED Task에 연결된 현재 분기 **CHECKED** PlanBlock도 정산 전까지 목록에 포함할 수 있으며 저장된 **display_title** 을 사용한다.

다음은 DB Enum으로 만들지 않는다.

```
FINALIZING
CHECK_IN_RESULT
NO_ACTIVE_CYCLE
NO_PLANS
IN_PROGRESS
DEADLINE_WARNING
```

마지막 날짜 EVENING에 PlanBlock이 없는 경우

해당 EVENING CheckIn 생성 안 함
→ planning_cycle ENDED
→ 남은 ACTIVE Task CANCELLED

그 후:

이전에 정산된 미확인 CheckIn 있음
→ 가장 최근 결과 하나를 CHECK_IN_RESULT로 표시

미확인 CheckIn 없음
→ NO_ACTIVE_CYCLE

결과 확인 시 화면에 표시된 CheckIn과 그보다 오래된 미확인 CheckIn만 함께 확인한다. 화면 표시 이후 생성된 더 최근 결과는 이번 확인 요청에 포함하지 않는다.

18. 마감 임박 경고 계산

마감 임박 경고는 홈 화면 상태가 아니라 별도 **blockingNotice** 로 계산한다.

경고 대상 조건

각 Task가 다음 조건을 모두 만족하면 **blockingNotice.items** 에 포함한다.

```
Task.status = ACTIVE
deadline_at IS NOT NULL
remaining_minutes > 0
```

```
deadline_warning_acknowledged_at IS NULL
effective_remaining_minutes > available_minutes
```

배치용 마감

```
cycle_end_at
= end_date 다음 날 04:00 Asia/Seoul
```

```
CASE
  WHEN deadline_at IS NULL THEN NULL
  WHEN deadline_at < cycle_end_at
    THEN deadline_at
  ELSE cycle_end_at
END
```

배치 가능 시간

```
available_minutes
= 지금부터 planning_deadline_at까지 남은 실제 분기 용량
- 고정 일정과 겹치는 시간
- 다른 Task에 이미 배정된 PlanBlock 시간
```

경고 대상 Task 자신에게 이미 배정된 PlanBlock 시간은 `available_minutes` 에서 다시 차감하지 않는다.

필요 시간

```
required_minutes
= effective_remaining_minutes
```

부족 시간

```
shortage_minutes
= required_minutes - available_minutes
```

서버 반환 구조

경고 대상이 한 개이든 여러 개이든 `items` 배열을 사용한다.

```
{
  "homeMode": "IN_PROGRESS",
  "blockingNotice": {
    "type": "DEADLINE_WARNING",
    "items": [
      {
        "taskId": "task-id-1",
        "title": "자료구조 과제",
        "deadlineAt": "2026-07-31T23:59:59+09:00",
        "requiredMinutes": 180,
        "availableMinutes": 120,
        "shortageMinutes": 60
      },
      {
        "taskId": "task-id-2",
        "title": "영어 과제",
        "deadlineAt": "2026-08-01T23:59:59+09:00",
        "requiredMinutes": 120,
        "availableMinutes": 60,
        "shortageMinutes": 60
      }
    ]
  }
}
```

경고 대상이 없으면 `blockingNotice` 는 `null` 이거나 응답에서 생략할 수 있다.

단일 `blockingNotice.task` 구조는 사용하지 않는다.

정렬

`items` 의 순서는 서버에서 고정한다.

1. `deadline_at`이 빠른 Task
2. 마감이 같으면 `shortage_minutes`가 큰 Task
3. 그래도 같으면 `created_at`이 빠른 Task
4. 마지막 tie-breaker는 Task.id

일괄 확인

```
POST /tasks/deadline-warnings/acknowledge
```

```
{
  "taskIds": ["task-id-1", "task-id-2"]
}
```

처리:

```
화면에 표시된 모든 Task의 deadline_warning_acknowledged_at 일괄 저장
→ GET /home/current
→ 최신 homeMode와 blockingNotice 재계산
```

기존 단일 Task API는 사용하지 않는다.

```
POST /tasks/{taskId}/deadline-warning/acknowledge
→ 제거
```

19. 분기 정산 처리

```
cycle_end_at
= planning_cycle.end_date 다음 날 04:00 Asia/Seoul
```

19-0. 정산 실행 정책

MVP에서도 분기 종료 시각에 서버 정산 Worker를 실행한다.

```
MORNING
→ 같은 날짜 12:00

AFTERNOON
→ 같은 날짜 18:00

EVENING
→ 다음 날 04:00에 전날 EVENING
```

정시 Worker는 앱 실행 여부와 관계없이 동작한다.

`GET /bootstrap`, `GET /home/current` 는 정산을 실행하지 않고 DB의 현재 상태만 반환한다.

서버 중단·Worker 실패를 대비한 장애 복구는 서버 내부 Worker가 담당한다.

서버 시작 시 복구 스캔
각 정시 Worker 실행 시 복구 스캔
finalized_at = NULL인 CheckIn 재처리
종료 시각이 지났지만 CheckIn이 없는 분기 탐지

복구 대상이 여러 개면 시간 순서대로 처리한다. 이 순서는 정상 사용자 흐름의 결과 표시 순서와 무관하다.

중복 정산 방지:

UNIQUE(plan_cycle_id, check_date, period)
CheckIn 행 잠금
planning_cycle 및 대상 PlanBlock 행 잠금
동일 분기 advisory lock
DB 트랜잭션

19-1. 정산 대상 확인

정시 Worker 또는 서버 내부 장애 복구 Worker은 다음 순서로 대상 분기를 확인한다.

1. 정산 대상 논리 날짜와 분기를 계산한다.
2. 해당 날짜가 planning_cycle.start_date ~ end_date에 포함되고
planning_cycle.status = ACTIVE인 cycle을 조회한다.
3. 해당 cycle·날짜·분기에 이미 CheckIn이 있는지 확인한다.
4. finalized_at IS NOT NULL인 CheckIn이 있으면
이미 정산이 끝난 분기이므로 건너뛰다.
5. finalized_at IS NULL인 CheckIn이 있으면
새 CheckIn을 만들지 않고 기존 정산을 복구한다.
6. CheckIn이 없으면 해당 분기의
PLANNED·CHECKED PlanBlock 개수를 확인한다.

PlanBlock이 0개이고 마지막 날짜 EVENING이 아닌 경우

CheckIn을 생성하지 않는다.
결과 화면을 생성하지 않는다.
기존 미래 PlanBlock은 그대로 유지한다.
해당 분기 정산을 종료한다.

PlanBlock이 0개이고 마지막 날짜 EVENING인 경우

하나의 트랜잭션에서 처리한다.

CheckIn을 생성하지 않는다.

```
remaining_minutes > 0인 ACTIVE Task
→ status = CANCELLED
→ cancelled_at = cycle_end_at

planning_cycle
→ status = ENDED
→ ended_at = cycle_end_at
```

상태와 연관 시각은 각각 같은 UPDATE에서 변경한다.

PlanBlock이 1개 이상인 경우

다음 CheckIn 정산 흐름을 실행한다.

19-2. FINALIZING 시작

해당 분기의 CheckIn이 아직 없으면 먼저 생성한다.

중복 Worker가 동시에 실행될 수 있으므로

`UNIQUE(plan_cycle_id, check_date, period)` 를 기준으로

중복 생성이 발생하지 않게 처리한다.

1. 해당 cycle·날짜·분기에 advisory lock 획득을 시도한다.
2. check_ins 행을 finalized_at = NULL로 생성한다.
3. 이미 같은 CheckIn이 존재하면 새로 만들지 않고 기존 행을 사용한다.
4. CheckIn 생성 작업을 먼저 커밋한다.
5. 앱이 홈 상태를 조회하면 finalized_at = NULL을 보고 FINALIZING을 표시한다.
6. 실제 정산 작업을 이어서 실행한다.

정시 Worker는 프론트의 폴링 여부와 관계없이 계속 실행된다.

프론트의 `GET /home/current` 는 정산 상태를 읽기 위한 요청이며,

정시 Worker가 정상 동작한 경우 정산 시작 자체를 프론트 요청에 의존하지 않는다.

19-3. 공통 정산 트랜잭션

정산의 논리적 종료 시각은 다음과 같이 계산한다.

```
MORNING
→ check_date 12:00
```


AFTERNOON

→ check_date 18:00

EVENING

→ check_date 다음 날 04:00

Worker가 장애 복구로 늦게 처리하더라도 정산으로 완료된 Task의 `completed_at`에는 위 논리적 종료 시각을 사용한다.

1. 해당 CheckIn 행을 `SELECT ... FOR UPDATE`로 잠근다.
2. `finalized_at IS NOT NULL`이면 이미 정산이 끝났으므로 종료한다.
3. 해당 분기의 PlanBlock 행을 잠근다.
4. PlanBlock 상태와 `check_in_id`를 같은 UPDATE에서 변경한다.

CHECKED

→ status = COMPLETED

→ check_in_id = 현재 CheckIn ID

→ checked_at은 기존 값 유지

PLANNED

→ status = NOT_DONE

→ check_in_id = 현재 CheckIn ID

→ checked_at은 NULL 유지

5. `total_plan_count`와 `completed_plan_count`를 계산한다.
6. Task별 완료 시간을 합산해 새로운 남은 시간을 계산한다.

```
new_remaining_minutes
= GREATEST(
    0,
    remaining_minutes - 해당 Task의 완료된 PlanBlock 시간 합
)
```

7. `remaining_minutes`와 Task 상태를 같은 UPDATE에서 변경한다.

기존 status = ACTIVE

AND new_remaining_minutes = 0

→ remaining_minutes = 0

→ status = COMPLETED

→ completed_at = 해당 분기의 논리적 종료 시각

→ cancelled_at = NULL 유지

```

기존 status = ACTIVE
AND new_remaining_minutes > 0

→ remaining_minutes = new_remaining_minutes
→ status = ACTIVE 유지
→ completed_at = NULL 유지
→ cancelled_at = NULL 유지

```

```

기존 status = CANCELLED

→ remaining_minutes = new_remaining_minutes
→ status = CANCELLED 유지
→ completed_at = NULL 유지
→ 기존 cancelled_at 유지

```

8. 점수를 계산한다.

```

score
= ROUND(
    completed_plan_count
    / total_plan_count
    × 100
)

```

9. `total_plan_count`, `completed_plan_count`, `score` 를 저장한다.

재계획과 cycle 종료 처리가 끝날 때까지 `finalized_at` 은 NULL로 유지한다.

19-4. 마지막 날짜 EVENING이 아닌 경우

10. 아직 시작하지 않은 미래 PLANNED PlanBlock을 삭제한다.
11. 변경된 Task.remaining_minutes와 고정 일정을 기준으로 남은 미래 분기의 PlanBlock을 다시 생성한다.
12. 배치하지 못한 시간을 계산한다.
13. 다음 값을 같은 UPDATE에서 저장한다.

```

replan_unplaced_minutes
replanned_at
finalized_at

```

14. 정산 트랜잭션을 커밋한다.

정산 완료 후:

```
finalized_at IS NOT NULL
AND result_acknowledged_at IS NULL
→ CHECK_IN_RESULT
```

19-5. 마지막 날짜 EVENING인 경우

미래 분기 재배치를 수행하지 않는다.

10. remaining_minutes > 0인 ACTIVE Task를 모두 CANCELLED로 변경한다.

```
status = CANCELLED
cancelled_at = cycle_end_at
```

11. planning_cycle을 ENDED로 변경한다.

```
status = ENDED
ended_at = cycle_end_at
```

12. CheckIn에 다음 값을 같은 UPDATE에서 저장한다.

```
replan_unplaced_minutes = 0
finalized_at = 현재 정산 완료 시각
```

13. 정산 트랜잭션을 커밋한다.

Task와 cycle의 상태 및 연관 시각은 각각 같은 UPDATE에서 변경한다.

정산 완료 후:

```
CHECK_IN_RESULT 표시
→ 사용자가 결과 확인
→ NO_ACTIVE_CYCLE
```

19-6. 정산 중 오류 복구

CheckIn 생성은 먼저 커밋되고 실제 정산은 뒤에서 실행되므로, 정산 도중 실패하면 `finalized_at = NULL` 인 행이 남을 수 있다.

실제 정산 트랜잭션이 실패하면 해당 트랜잭션을 롤백한다.

```
PlanBlock 상태 변경 롤백
Task remaining_minutes·상태 변경 롤백
미래 PlanBlock 삭제·재생성 롤백
planning_cycle 종료 변경 롤백
CheckIn 최종 결과 저장 롤백
```

`finalized_at = NULL` 인 CheckIn은 삭제하지 않는다.

다음 서버 내부 복구 Worker가 동일 CheckIn을 잠그고 다시 정산한다.

서버 시작 시 복구 스캔
다음 정시 Worker 실행 시 복구 스캔
운영자가 명시적으로 실행하는 복구 작업

조회 API는 복구를 직접 수행하지 않는다.

GET /bootstrap
GET /home/current
→ 현재 `finalized_at` 상태만 조회
→ FINALIZING 또는 완료 상태 반환

복구 시 새 CheckIn을 만들지 않는다.

UNIQUE(plan_cycle_id, check_date, period)
→ 중복 CheckIn 방지

동일 분기 advisory lock
→ 동시 정산 방지

CheckIn 행 잠금
→ 완료된 정산 중복 반영 방지

복구 대상이 여러 개면 Task 남은 시간과 재계획 순서를 보존하기 위해 시간상 이전 대상부터 처리한다.

이 복구는 비정상 서버 상황에 대한 안전장치이며 정상 화면 흐름은 매 분기 정시 자동 정산이다.

20. 계획 실행 트랜잭션

계획 실행은 동일한 ``solar_requests.id``를 실행 단위로 사용한다.

별도의 실행용 요청이나 실행 결과 테이블을 만들지 않는다.

최초 실행
→ POST /solar/requests/{requestId}/execute

실패 후 재시도
→ POST /solar/requests/{requestId}/retry

같은 `requestId`에 대한 실행 작업은 동시에 하나만 수행할 수 있다.

20-1. 실행 요청 상태 1차 확인

서버는 대상 `solar_requests` 를 조회해 요청 사용자, `requestId`, `purpose`, 현재 상태를 확인한다.

최초 실행

```
POST /solar/requests/{requestId}/execute
```

실행 가능 상태
→ FINAL_REVIEW

재시도

```
POST /solar/requests/{requestId}/retry
```

실행 가능 상태
→ FAILED

실행하지 않는 경우

이미 EXECUTING
→ 새로운 실행 작업 등록 안 함
→ 현재 EXECUTING 상태 반환

이미 COMPLETED
→ 새로운 실행 작업 등록 안 함
→ 현재 COMPLETED 상태 반환

FAILED 상태에서 일반 execute 요청
→ 새로운 실행 작업 등록 안 함
→ 현재 FAILED 상태 반환
→ retry API로만 다시 실행 가능

COLLECTING
CHANGE_CONFIRMATION
CHANGE_INPUT

→ 실행 불가
→ 현재 상태 유지
→ 실행 작업 등록 안 함

이 단계는 1차 확인이다. purpose별 사전 검증 이후 **EXECUTING** 전환 트랜잭션에서 요청 행을 잠그고 상태를 다시 확인한다.

20-2. **ACTIVE_CYCLE** 실행 전 정산 상태 확인

분기 정산은 정시 Worker가 담당한다. 계획 실행 API가 과거 분기를 직접 정산하지 않는다.

purpose = ACTIVE_CYCLE 요청은 **EXECUTING** 전환 전에 최신 상태를 확인한다.

1. 요청에 연결된 **planning_cycle** 조회
2. 해당 사용자의 **finalized_at = NULL** CheckIn 존재 여부 확인
3. 종료 시각이 지났지만 정산 완료가 확인되지 않은 분기 존재 여부 확인
4. Task·PlanBlock·고정 일정 최신 상태 조회

정산 또는 장애 복구가 진행 중이면:

계획 실행 시작 안 함
solar_requests.status 기존 상태 유지
execution_attempt_count 증가 없음
execution_started_at 변경 없음
BackgroundTasks 등록 없음

서버는 내부 복구 Worker가 처리할 수 있도록 복구 신호를 남기거나 작업을 enqueue할 수 있다. 사용자에게는 정산 완료 후 다시 시도하도록 안내한다.

정산이 끝난 뒤 다시 시도해 주세요.

이 검증은 계획 실행과 정산이 동시에 같은 Task·PlanBlock을 변경하는 충돌을 막기 위한 것이다.

NEW_CYCLE은 수정 대상 기존 cycle이 없지만, 실행 직전에 사용자 ACTIVE cycle이 없는지 반드시 재검증한다.

20-3. 대상 재검증

정산 진행 상태가 아니어도 채팅 시작 이후 데이터가 바뀔 수 있으므로 최신 DB 기준으로 대상을 다시 검증한다.

planning_cycle

planning_cycle 존재
요청 사용자와 소유자 일치
status = ACTIVE

Task

수정·삭제 대상:

```
Task 존재
Task.user_id = 요청 사용자
Task.plan_cycle_id = 대상 cycle
Task.status = ACTIVE
```

다음 상태는 수정·삭제하지 않는다.

```
COMPLETED
CANCELLED
```

고정 일정

수정·삭제 대상:

```
fixed_schedule 존재
user_id = 요청 사용자
plan_cycle_id = 대상 cycle
start_at > 현재 시각
```

다음 일정은 변경하지 않는다.

```
이미 종료됨
현재 진행 중
과거 일정
```

PlanBlock

변경하지 않는 대상:

```
과거 COMPLETED
과거 NOT_DONE
현재 분기 CHECKED
과거 분기의 모든 PlanBlock
```

재계획 대상:

```
현재 분기의 미체크 PLANNED
미래 분기의 PLANNED
```

20-4. 실행 전 검증 실패

사전 검증에서 문제가 발견되면 실제 실행을 시작하지 않는다.

최초 execute

```
solar_requests.status = FINAL_REVIEW 유지  
execution_attempt_count 증가 없음  
execution_started_at 변경 없음  
BackgroundTasks 등록 없음  
Task·고정 일정·PlanBlock 변경 없음
```

사용자는 화면의 **[수정]**으로 **CHANGE_INPUT**에 돌아가거나 요청을 삭제할 수 있다.

retry

```
solar_requests.status = FAILED 유지  
execution_attempt_count 증가 없음  
execution_started_at 변경 없음  
BackgroundTasks 등록 없음  
Task·고정 일정·PlanBlock 변경 없음
```

사용자는 **[다시 시도]** 또는 **[요청 취소]**만 사용할 수 있다.

예:

```
대상 cycle이 이미 ENDED  
수정 대상 Task가 COMPLETED  
고정 일정 시작 시각이 이미 지남  
정산 또는 장애 복구가 진행 중
```

실행 전 검증 실패는 실제 실행 worker 오류가 아니므로 새롭게 **FAILED**로 전환하지 않는다.

20-5. **EXECUTING** 상태 전이

사전 검증이 모두 성공하면 짧은 DB 트랜잭션을 시작한다.

요청 행 잠금과 최종 재검증

1. 대상 solar_requests 행을 SELECT ... FOR UPDATE로 잠근다.
2. 요청 상태가 여전히 실행 가능한지 확인한다.

```
최초 execute  
→ FINAL_REVIEW
```

```
retry
```


→ FAILED

3. purpose별 핵심 조건을 다시 확인한다.

NEW_CYCLE

→ 사용자 ACTIVE planning_cycle 없음

ACTIVE_CYCLE

→ 대상 planning_cycle.status = ACTIVE

→ 요청 사용자와 cycle 소유자 일치

4. 검증 성공 시에만 조건부 UPDATE 수행

최초 실행 상태 변경

```
UPDATE solar_requests
SET
    status = 'EXECUTING',
    execution_started_at = now(),
    execution_attempt_count = execution_attempt_count + 1,
    executed_at = NULL,
    execution_result = NULL,
    error_code = NULL,
    error_message = NULL,
    updated_at = now()
WHERE
    id = :request_id
    AND user_id = :user_id
    AND status = 'FINAL_REVIEW'
RETURNING id, status;
```

재시도 상태 변경

```
UPDATE solar_requests
SET
    status = 'EXECUTING',
    execution_started_at = now(),
    execution_attempt_count = execution_attempt_count + 1,
    executed_at = NULL,
    execution_result = NULL,
    error_code = NULL,
    error_message = NULL,
    updated_at = now()
```

```
WHERE
    id = :request_id
    AND user_id = :user_id
    AND status = 'FAILED'
RETURNING id, status;
```

다음 컬럼은 같은 UPDATE에서 변경한다.

```
status = EXECUTING
execution_started_at = 현재 시각
execution_attempt_count = 기존 값 + 1
executed_at = NULL
execution_result = NULL
error_code = NULL
error_message = NULL
updated_at = 현재 시각
```

실행 작업 등록

```
조건부 UPDATE 결과 1행
→ EXECUTING 전환 성공
→ 트랜잭션 커밋
→ FastAPI BackgroundTasks에 실제 실행 작업 등록
```

```
조건부 UPDATE 결과 0행
→ 다른 호출이 먼저 상태를 변경했거나 현재 실행 불가
→ 새 실행 작업 등록 안 함
→ 현재 요청 상태 반환
```

같은 `requestId` 로 `execute` 또는 `retry`가 중복 호출되어도 조건부 UPDATE에 성공한 호출만 실행 작업을 등록한다.

20-6. 실행 worker 공통 규칙

일반 BackgroundTasks 실행과 서버 재시작 후 복구 실행 모두

같은 실행 worker를 사용한다.

실행 worker는 작업 시작 전에 `requestId` 를 기준으로

PostgreSQL session-level advisory lock 획득을 시도한다.

```
lock 획득 성공
→ 해당 요청 실행

lock 획득 실패
```

→ 동일 요청을 다른 worker가 실행 중
→ 현재 worker는 실행하지 않고 종료

advisory lock은 실제 실행이 성공하거나 실패할 때까지 유지한다.

정상 종료
→ lock 해제

worker 비정상 종료
→ DB 연결 종료
→ session-level advisory lock 자동 해제

따라서 동일한 `requestId`에 대한 실제 실행 worker는

동시에 하나만 존재할 수 있다.

실행 worker는 대상 `solar_requests`를 다시 조회하고

다음을 확인한다.

status = EXECUTING
requestId 일치
요청 사용자 일치

조건을 만족하지 않으면 실제 데이터 변경을 수행하지 않는다.

20-7. NEW_CYCLE 실행

`purpose = NEW_CYCLE`인 요청은 하나의 실행 트랜잭션에서 처리한다.

일반적인 ACTIVE cycle 충돌은 `EXECUTING` 전 사전 검증에서 차단한다.

ACTIVE cycle이 사전 검증에서 발견됨

최초 execute
→ FINAL_REVIEW 유지

retry
→ FAILED 유지

공통
→ 실제 worker 등록 안 함

실행 worker에서도 동시성 경쟁을 막기 위해 ACTIVE cycle 존재 여부를 최종 확인한다.

사전 검증 이후 다른 트랜잭션이 ACTIVE cycle을 생성했거나
uq_planning_cycles_one_active_per_user 제약과 충돌

- 이미 EXECUTING으로 시작된 실제 실행 단계의 충돌
- 전체 실행 트랜잭션 롤백
- solar_requests를 FAILED로 저장

실행 순서

1. 사용자 ACTIVE cycle 부재 최종 확인
2. planning_cycle 생성
3. Task 생성
4. 고정 일정 생성
5. 생성된 Task와 고정 일정을 기준으로
7일 범위 PlanBlock 생성
6. 생성된 Task와 고정 일정
source_request_item_id 연결
7. 모든 solar_request_items를 EXECUTED로 변경하고
같은 UPDATE에서 executed_at 저장
8. 실행 결과 요약 생성
9. solar_requests를 COMPLETED로 변경하고
같은 UPDATE에서 다음 값 저장

plan_cycle_id = 생성한 cycle ID
executed_at
execution_result
error_code = NULL
error_message = NULL
updated_at
10. 전체 실행 트랜잭션 커밋

실행 결과 예시

```
{  
  "createdTaskCount": 2,  
  "updatedTaskCount": 0,  
  "cancelledTaskCount": 0,  
  "createdFixedScheduleCount": 1,  
  "updatedFixedScheduleCount": 0,
```

```
"deletedFixedScheduleCount": 0
}
```

`planning_cycle`, Task, 고정 일정, PlanBlock, `solar_request_items`, `solar_requests` 변경은 하나의 실행 트랜잭션으로 처리한다.

하나라도 실패하면 모두 롤백한다.

20-8. `ACTIVE_CYCLE` 실행

`purpose = ACTIVE_CYCLE` 인 요청은 다음 사전 검증이 완료된 이후에만 실행한다.

```
정산 또는 장애 복구가 진행 중이 아님
Task.remaining_minutes 최신 값 조회
PlanBlock 상태 최신 값 조회
planning_cycle ACTIVE 상태 재검증
수정·삭제 대상 유효성 재검증
```

실제 변경은 하나의 실행 트랜잭션에서 처리한다.

1. 대상 행 잠금 및 최종 검증

```
planning_cycle 행 잠금
수정·삭제 대상 Task 행 잠금
수정·삭제 대상 고정 일정 행 잠금
재계획 대상 PlanBlock 행 잠금
```

잠금 후에도 다음을 다시 확인한다.

```
planning_cycle.status = ACTIVE
Task.status = ACTIVE
고정 일정 start_at > 현재 시각
요청 상태 = EXECUTING
```

2. Task 변경

```
Task 추가
Task 수정
Task 취소
```

Task 취소 요청은 다음과 같이 처리한다.

```
Task.status = CANCELLED
Task.cancelled_at 저장
```

현재 분기의 미체크 PLANNED PlanBlock 삭제
미래 분기의 PLANNED PlanBlock 삭제

과거 COMPLETED·NOT_DONE PlanBlock 유지
현재 분기의 CHECKED PlanBlock 유지

3. 고정 일정 변경

미래 고정 일정 추가
미래 고정 일정 수정
미래 고정 일정 삭제

삭제 대상 미래 고정 일정은 물리 삭제한다.

과거 고정 일정 유지
현재 진행 중 고정 일정 유지
이미 종료된 고정 일정 유지

4. 기존 PlanBlock 정리

다음 PlanBlock은 유지한다.

과거 COMPLETED
과거 NOT_DONE
현재 분기 CHECKED

다음 PlanBlock은 삭제 후 재계산할 수 있다.

현재 분기의 미체크 PLANNED
미래 분기의 PLANNED

5. Task별 실제 남은 시간 계산

현재 분기에 아직 정산되지 않은 **CHECKED** PlanBlock 시간은

remaining_minutes 에 아직 반영되지 않았으므로 다음 값을 사용한다.

```
unfinalized_checked_minutes  
= 해당 Task의 현재 분기 CHECKED PlanBlock 시간 합
```

```
effective_remaining_minutes
= GREATEST(
    0,
    remaining_minutes - unfinalized_checked_minutes
)
```

현재 분기에 이미 **CHECKED** PlanBlock이 있는 Task는 같은 날짜·분기에 새 PlanBlock을 추가하지 않는다. 해당 Task의 남은 시간은 이후 미래 분기부터 배치한다.

6. PlanBlock 재계산

현재 CHECKED PlanBlock 시간은 고정
고정 일정 점유 시간 반영
현재 시각 이후 실제 사용 가능 시간 반영
각 분기 최대 240분 제한 반영
PlanBlock 정수 분 단위 반영
Task의 effective_remaining_minutes 반영

위 조건을 기준으로 현재 시점 이후의 PlanBlock을 다시 생성한다.

7. 요청 카드 실행 완료 처리

```
solar_request_items.status = EXECUTED
solar_request_items.executed_at = 현재 시각
```

상태와 실행 시각은 같은 UPDATE에서 변경한다.

8. 요청 실행 완료 처리

실행 결과 요약을 생성한 뒤 다음 값을 같은 UPDATE에서 변경한다.

```
solar_requests.status = COMPLETED
solar_requests.executed_at = 현재 시각
solar_requests.execution_result = 실행 결과
solar_requests.error_code = NULL
solar_requests.error_message = NULL
solar_requests.updated_at = 현재 시각
```

9. 트랜잭션 커밋

Task
고정 일정
PlanBlock

```
solar_request_items  
solar_requests
```

- 모든 변경 성공
- 전체 실행 트랜잭션 커밋

하나라도 실패하면 전체 실행 트랜잭션을 롤백한다.

20-9. 실행 성공과 상태 조회

실행 성공 시 다음 조건을 만족해야 한다.

```
solar_requests.status = COMPLETED  
execution_started_at IS NOT NULL  
execution_attempt_count >= 1  
executed_at IS NOT NULL  
execution_result IS NOT NULL  
error_code IS NULL  
error_message IS NULL
```

실행 성공 이후 결과 화면을 아직 확인하지 않았다면:

```
result_acknowledged_at IS NULL  
→ EXECUTION_SUCCESS
```

실행 성공으로 Task·고정 일정·PlanBlock 데이터가 변경되면 홈·캘린더·계획관리 조회 캐시를 무효화한다.

```
현재 포커스된 탭  
→ 해당 탭 API 즉시 재호출  
  
포커스되지 않은 탭  
→ 다음 탭 진입 시 API 재호출
```

다만 사용자가 보고 있는 화면을 강제로 `EXECUTION_SUCCESS` 로 전환하지 않는다.

EXECUTING 상태 조회 실패

```
GET /solar/requests/{requestId}/execution 실패  
→ solar_requests.status 변경 없음  
→ FAILED로 변경하지 않음  
→ 실행 worker 중단으로 판단하지 않음  
→ 같은 requestId 상태 다시 조회
```


네트워크 오류만으로 `EXECUTION_FAILED` 를 표시하지 않는다. 서버가 실제 `status = FAILED` 를 반환한 경우에만 실패 화면을 표시한다.

20-10. 실행 실패

실제 `EXECUTING` 상태로 변경된 이후

Task·고정 일정·PlanBlock 반영 중 오류가 발생하면 실행 실패로 처리한다.

데이터 변경 롤백

1. Task·고정 일정·PlanBlock 반영 트랜잭션 롤백
2. `solar_request_items`의 `EXECUTED` 변경 롤백
3. 생성 중이던 `planning_cycle`·Task·고정 일정·PlanBlock 롤백
4. 기존 실제 계획 데이터는 실행 전 상태로 유지

요청 실패 상태 저장

롤백 이후 별도 트랜잭션에서 다음 값을 같은 UPDATE로 변경한다.

```
solar_requests.status = FAILED
solar_requests.error_code = 실패 코드
solar_requests.error_message = 실패 안내 문구
solar_requests.executed_at = NULL
solar_requests.execution_result = NULL
solar_requests.updated_at = 현재 시각
```

다음 값을 유지한다.

```
execution_started_at
execution_attempt_count
```

실행 실패 시 새 `solar_request` 를 만들지 않는다.

기존 요청과 기존 `solar_request_items` 를 유지한다.

20-11. 다시 시도

실패한 요청은 같은 요청과 같은 카드를 재사용한다.

```
동일 solar_request 사용
동일 solar_request_items 사용
```

새 요청 생성 안 함
새 카드 생성 안 함

retry도 최초 실행과 동일한 공통 검증 및 purpose별 검증을 다시 수행한다.

1. 현재 status = FAILED인지 확인
2. purpose 확인
3. NEW_CYCLE
→ ACTIVE planning_cycle 부재 재검증
4. ACTIVE_CYCLE
→ 정산 진행 여부 확인
→ Task·PlanBlock 최신 상태 조회
→ cycle과 수정·삭제 대상 재검증
5. 검증 성공 시 FAILED → EXECUTING
6. execution_attempt_count 증가
7. execution_started_at을 재시도 시작 시각으로 갱신
8. executed_at 초기화
9. execution_result 초기화
10. error_code 초기화
11. error_message 초기화
12. 실행 worker 등록

검증 실패 시:

status = FAILED 유지
execution_attempt_count 증가 없음
execution_started_at 변경 없음
실행 worker 등록 없음

새 Task와 고정 일정의 중복 생성을 방지한다.

source_request_item_id UNIQUE
요청 행 잠금
조건부 상태 UPDATE
실행 트랜잭션
advisory lock

20-12. EXECUTING 복구

FastAPI 서버가 재시작되면 기존 `BackgroundTasks` 작업은

유지되지 않을 수 있다.

서버 시작 시 다음 복구 절차를 수행한다.

1. status = EXECUTING인 solar_requests 조회
2. 각 requestId를 기준으로 PostgreSQL session-level advisory lock 획득 시도
3. lock 획득에 성공한 worker만 복구 실행
4. lock 획득에 실패한 요청은 다른 worker가 실행 중인 것으로 판단하고 건너뛴
5. 동일 solar_requests와 solar_request_items를 사용해 실행 재개
6. 실행 성공 또는 실패 처리가 완료될 때까지 lock 유지
7. 처리 완료 후 lock 해제

복구 worker는 요청 데이터를 새로 만들지 않는다.

```
새 solar_request 생성 안 함
새 solar_request_items 생성 안 함
execution_attempt_count 추가 증가 안 함
execution_started_at 새로 변경 안 함
```

복구 실행은 이미 시작된 동일 실행 시도를 이어서 처리하는 것이므로

새로운 retry로 취급하지 않는다.

복구 실행도 일반 실행과 동일하게 다음 규칙을 사용한다.

```
source_request_item_id UNIQUE
solar_requests 행 잠금
planning_cycle 및 대상 행 잠금
DB 트랜잭션
session-level advisory lock
```

복구 worker가 비정상 종료되면

DB 연결 종료와 함께 advisory lock이 자동으로 해제된다.

다음 서버 시작 또는 복구 요청에서 동일 요청을 다시 복구할 수 있다.

20-13. 최종 실행 순서 요약

NEW_CYCLE

요청 상태 1차 확인

- ACTIVE cycle 부재 사전 검증
- 요청 행 잠금
- 상태와 ACTIVE cycle 부재 최종 재검증
- EXECUTING 전환
- 실행 worker 등록
- worker의 ACTIVE cycle 부재 최종 확인
- planning_cycle 생성
- Task 생성
- 고정 일정 생성
- PlanBlock 생성
- 카드 EXECUTED
- 요청 COMPLETED

ACTIVE_CYCLE

요청 상태 확인

- CheckIn 정산·장애 복구 진행 여부 확인
- 진행 중이면 실행 보류
- Task.remaining_minutes 최신 값 조회
- PlanBlock 상태 최신 값 조회
- cycle과 수정 대상 재검증
- EXECUTING 전환
- 실행 worker 등록
- Task·고정 일정 변경
- 현재 미체크·미래 PLANNED 재계산
- 카드 EXECUTED
- 요청 COMPLETED

실행 전 검증 실패

최초 execute

- FINAL_REVIEW 유지

retry

- FAILED 유지

공통

- 실행 횟수 증가 없음
- 실행 worker 등록 없음
- 실제 계획 데이터 변경 없음

실제 실행 중 오류

실제 데이터 변경 롤백

- 별도 트랜잭션으로 요청 FAILED
- 기존 요청과 카드 유지
- retry 가능

상태 조회 오류

EXECUTING 조회 실패
→ DB 상태 변경 없음
→ FAILED로 판단하지 않음
→ 다시 확인

중복 호출 또는 서버 재시작

조건부 상태 UPDATE
+ requestId advisory lock
+ source_request_item_id UNIQUE
+ 사용자당 ACTIVE cycle UNIQUE
+ DB 트랜잭션

→ 중복 실행·중복 생성·ACTIVE cycle 중복 방지

21. 수정·삭제 가능 범위

SOLAR를 통한 수정·삭제는 현재 및 미래 데이터에만 허용한다.

Task와 PlanBlock

수정·삭제 불가:

COMPLETED PlanBlock
NOT_DONE PlanBlock
과거 분기의 PlanBlock
현재 분기의 CHECKED PlanBlock
COMPLETED Task
CANCELLED Task

수정·삭제 가능:

현재 분기의 PLANNED PlanBlock
미래 분기의 PLANNED PlanBlock
ACTIVE Task의 현재·미래 계획

Task 삭제 요청:

Task.status = CANCELLED
Task.cancelled_at 저장

현재 분기의 미체크 PLANNED PlanBlock 삭제
미래 PLANNED PlanBlock 삭제
현재 분기의 CHECKED PlanBlock은 유지
과거 COMPLETED·NOT_DONE PlanBlock 유지

현재 분기의 유지된 CHECKED PlanBlock

- 분기 종료 전 홀에 표시 가능
- 정산 시 COMPLETED로 확정
- CheckIn 결과와 과거 캘린더에 기록

이후 현재 분기 정산에서 CHECKED 시간이 반영되어
remaining_minutes = 0이 되더라도 Task는 CANCELLED 상태를 유지한다.

Task 내용 수정:

Task 정보 수정

과거 COMPLETED·NOT_DONE PlanBlock의 상태·시간·점수는 변경하지 않음
현재 분기의 미체크 PLANNED PlanBlock과 미래 PLANNED PlanBlock만 재계산

고정 일정

수정·삭제 불가:

이미 종료된 고정 일정
현재 진행 중인 고정 일정
과거 고정 일정

수정·삭제 가능:

start_at이 현재 시각보다 뒤인 미래 고정 일정

미래 고정 일정 삭제:

대상 fixed_schedules 행 물리 삭제
과거 고정 일정에는 영향 없음

22. 캘린더 조회 정책

캘린더는 cycleId 없이 날짜 범위로 조회한다.

GET /calendar?from=...&to=...

캘린더는 조회 전용이며 PlanBlock 체크·체크 해제를 제공하지 않는다.

PlanBlock 카드 표시

홈·CHECK_IN_RESULT·캘린더는 모두 `plan_blocks.display_title` 을 기본 카드 문구로 사용한다.

자료구조 2문제
영단어 암기
대학영어 발표 PPT 3장 만들기
산책 10분

- `display_title` 에 분량이나 시간이 포함돼 있으면 한 줄 문구 안에서 함께 표시한다.
- 분량·시간을 별도 라벨로 강제 분리하지 않는다.
- 캘린더 조회 시 Task의 현재 제목과 분량을 다시 조합하지 않는다.
- 과거 COMPLETED PlanBlock은 당시 저장된 `display_title` 을 유지한다.
- CANCELLED Task에 연결된 과거 COMPLETED PlanBlock도 동일하게 `display_title` 로 표시한다.

과거 날짜·분기

표시:

COMPLETED PlanBlock
과거 고정 일정
finalized_at IS NOT NULL인 CheckIn 점수

숨김:

NOT_DONE PlanBlock

CheckIn의 `result_acknowledged_at` 여부는 캘린더 표시 조건이 아니다.

현재 날짜·분기

표시:

PLANNED PlanBlock
CHECKED PlanBlock
고정 일정

체크 기능:

없음

미래 날짜·분기

표시:

PLANNED PlanBlock
고정 일정

미표시:

점수
체크 기능
상단 완료 개수
상단 미완료 개수
진행률

재계획 후에는 삭제된 과거 미래 블록이 아니라 현재 DB에 남은 최신 **PLANNED** PlanBlock과 새 **display_title** 을 반환한다.

자정을 넘는 고정 일정

고정 일정은 원본 한 행을 유지한다.

금요일 22:00 ~ 토요일 02:00

위 일정은 **resolvePlanDate** 와 **resolvePeriod** 기준으로 전날 **EVENING** 에 묶인다.

금요일 22:00
→ 금요일 EVENING

토요일 02:00
→ 논리 날짜 금요일 EVENING

하나의 일정이 04:00 이후까지 이어져 여러 논리 날짜·분기에 걸치면 캘린더 API가 조회 범위와 겹치는 구간을 각 날짜·분기에 표시할 수 있다. DB 원본 행을 분할 저장하지 않는다.

0점

0점은 CheckIn이 실제 존재하고 정산이 완료된 분기에만 표시한다.


```
total_plan_count > 0
completed_plan_count = 0
score = 0
finalized_at IS NOT NULL
```

CheckIn이 없으면 0점을 추론해 표시하지 않는다.

빈 화면

DB에서는 캘린더 빈 상태 원인을 나누지 않는다.

```
{
  "planBlocks": [],
  "fixedSchedules": [],
  "checkIns": []
}
```

항목이 모두 없으면 모든 경우 같은 빈 화면을 표시한다.

이 날짜에는 표시할 계획이 없어요.

계획관리 이동 버튼이나 새 계획 시작 버튼은 표시하지 않는다.

23. 최종 제외 컬럼·테이블

사용자

```
user_profiles.email
provider
social_id
password
is_active
deactivated_at
로그인 입력 오류 저장 컬럼
회원가입 입력 오류 저장 컬럼
약관 동의 이력 테이블
```

반복

```
recurrence_group_id
recurrence_index
```

```
recurrence_rule
repeat_days
target_scope
SINGLE
RECURRENCE_GROUP
```

문서

```
documents
document_id
amount_pdf_url
parse_status
extract_status
parsed_output
extracted_output
```

cycle 교체

```
replaced_plan_cycle_id
기존 cycle 종료 후 새 cycle 생성 관련 상태
```

직접 수정

Task 남은 시간 직접 수정 이력 테이블
remaining_minutes 전용 수정 API 데이터

화면 전용 상태

다음은 프론트 계산값 또는 API 응답값이며 DB Enum·테이블·컬럼으로 추가하지 않는다.

```
NEW_CYCLE_ENTRY
ACTIVE_CYCLE_ENTRY
EXECUTION_SUCCESS
EXECUTION_FAILED
FINALIZING
CHECK_IN_RESULT
NO_ACTIVE_CYCLE
NO_PLANS
IN_PROGRESS
DEADLINE_WARNING
blockingNotice
blockingNotice.items
currentQuestion
quickReplies
inputPlaceholder
pendingItemId
```

하단 탭 표시 여부
일반 오류 문구
진행 상태 오류 문구
입력칸별 오류 문구

`blockingNotice.items` 는 `tasks` 조회 결과로 계산하고, 질문 데이터는 `solar_request_items.pending_question` 과 `solar_messages.metadata` 에서 구성한다.

24. Supabase RLS·접근 경계

모바일 앱은 Supabase Auth를 로그인에 사용하고, 실제 계획 데이터 변경은 FastAPI를 통해 수행한다.

원칙

- 모든 앱 테이블에서 RLS를 활성화한다.
- 모바일의 `authenticated` 세션이 다른 사용자의 행을 읽거나 변경할 수 없게 한다.
- 정산 Worker와 계획 실행 Worker는 신뢰된 백엔드 연결 또는 Supabase service role을 사용한다.
- service role 키는 모바일 앱에 포함하지 않는다.
- FastAPI는 사용자 JWT를 검증하고 요청 사용자 ID와 행의 `user_id` 를 다시 확인한다.
- RLS만 믿지 않고 복합 FK와 서버 검증을 함께 사용한다.

최소 정책 형태

`user_profiles` :

```
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY user_profiles_select_own
ON user_profiles
FOR SELECT
TO authenticated
USING (id = auth.uid());
```

`user_id` 가 있는 앱 테이블:

```
ALTER TABLE planning_cycles ENABLE ROW LEVEL SECURITY;
ALTER TABLE solar_requests ENABLE ROW LEVEL SECURITY;
ALTER TABLE solar_request_items ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE solar_messages ENABLE ROW LEVEL SECURITY;  
ALTER TABLE tasks ENABLE ROW LEVEL SECURITY;  
ALTER TABLE fixed_schedules ENABLE ROW LEVEL SECURITY;  
ALTER TABLE plan_blocks ENABLE ROW LEVEL SECURITY;  
ALTER TABLE check_ins ENABLE ROW LEVEL SECURITY;
```

직접 Supabase 조회를 허용하는 경우 각 테이블의 SELECT 정책은 다음 형태로 제한한다.

```
USING (user_id = auth.uid())
```

INSERT·UPDATE·DELETE를 모바일에서 직접 수행하지 않는 구조라면 해당 정책을 만들지 않고 FastAPI만 변경 권한을 가진다.

25. 화면 흐름 더블체크 결과

최종 화면 흐름과 DB 구조를 다시 대조한 결과다.

인증·온보딩

화면:

Supabase 세션 → onboarding_completed → 닉네임 생성 또는 초기 화면

DB:

auth.users

user_profiles.nickname

user_profiles.onboarding_completed

일치한다.

계획관리 9개 화면 모드

DB 상태 :

COLLECTING

CHANGE_CONFIRMATION

CHANGE_INPUT

FINAL_REVIEW

EXECUTING

COMPLETED

FAILED

계산 화면 :

NEW_CYCLE_ENTRY

ACTIVE_CYCLE_ENTRY

```
EXECUTION_SUCCESS
EXECUTION_FAILED
```

중간 질문 Route나 불필요한 DB Enum을 추가하지 않았다.

PlanBlock 자연스러운 한 줄 문구

```
구조화 데이터:
tasks.title
tasks.amount_text
plan_blocks.allocated_amount_text
plan_blocks.allocated_minutes
```

```
화면 확정 문구:
plan_blocks.display_title
```

- 홈·결과·캘린더가 같은 문구를 사용한다.
- 상태만 변경될 때 문구를 유지한다.
- 재계획으로 새 블록이 생길 때만 새 문구를 확정한다.
- 과거 COMPLETED·NOT_DONE 문구를 현재 Task 제목으로 덮어쓰지 않는다.
- **산책 10분**을 지원하도록 30분 배수 제약을 제거했다.

화면 흐름과 일치한다.

고정 일정 자정 초과

```
start_at
end_at
→ 서로 다른 달력 날짜 허용
→ 원본 한 행 유지
```

자정을 넘는 일정 지원과 일치한다.

홈 상태 우선순위

```
FINALIZING
→ CHECK_IN_RESULT
→ DEADLINE_WARNING
→ NO_ACTIVE_CYCLE
→ NO_PLANS
→ IN_PROGRESS
```

화면 흐름과 일치한다.

최신 CheckIn 결과

표시:
ORDER BY finalized_at DESC, id DESC
LIMIT 1

확인:
표시 대상 + 그보다 오래된 미확인 결과

제외:
화면 표시 뒤 생성된 더 최근 결과

화면 흐름과 일치한다.

CheckIn 결과 목록

COMPLETED
NOT_DONE
→ 홈 결과에 모두 표시
→ display_title 사용

캘린더:
COMPLETED만 목록 표시
NOT_DONE 숨김

화면 흐름과 일치한다.

분기 자동 정산

12:00
18:00
다음 날 04:00
→ 서버 Worker 자동 정산
→ 앱 접속 여부와 무관

`finalized_at = NULL` 은 FINALIZING 복원에 사용한다. 요청 기반 복구는 서버 장애 시 안전장치일 뿐 정상 화면 흐름의 트리거가 아니다.

마감 임박 경고

requiredMinutes
= effectiveRemainingMinutes

blockingNotice.items
→ 모든 대상 Task

```
acknowledge
→ taskIdIds 일괄 처리
```

화면 흐름과 일치한다.

EXECUTING·오류

```
EXECUTING 조회 실패
→ status 유지
→ FAILED로 변경하지 않음

FINALIZING 조회 실패
→ finalized_at 유지
→ 정산 실패로 단정하지 않음
```

화면 흐름과 일치한다.

캘린더

```
과거:
COMPLETED + 점수 + 고정 일정

현재:
PLANNED + CHECKED + 고정 일정

미래:
PLANNED + 고정 일정

빈 화면:
세 배열 모두 빈 배열
```

화면 흐름과 일치한다.

최종 테이블 수

```
Supabase Auth
→ auth.users
```

```
앱 DB
→ 9개 테이블
```

```
1. user_profiles
2. planning_cycles
3. solar_requests
4. solar_request_items
5. solar_messages
```

```
6. tasks
7. fixed_schedules
8. plan_blocks
9. check_ins
```

추가 테이블 없이 최종 화면 흐름을 구현할 수 있다.

이 문서를 이음 MVP의 최종 DB 구조 기준으로 동결한다.